

MONOPOLY.LK

A Sri Lanka themed Monopoly simulation in C

Complete Source Code Reference

Every file, function, variable, macro, enum and struct field

Generated from the actual source code

Table of Contents

1	How to Use This Reference.....	3
	Files covered.....	4
2	Compile-Time Constants (types.h #define).....	4
3	Enums and Data Structures (types.h).....	7
	3.1 enum SquareType.....	8
	3.2 enum PropertyGroup.....	8
	3.3 enum PlayerType.....	8
	3.4 enum InsuranceType.....	8
	3.5 enum DisasterType.....	8
	3.6 enum ModifierType.....	9
	3.7 enum ModifierSource.....	9
	3.8 struct Loan.....	9
	3.9 struct Property.....	9
	3.10 struct Square.....	10
	3.11 struct Player.....	10
	3.12 struct ActiveModifier.....	10
	3.13 struct Economy.....	11
	3.14 struct GameState.....	11
4	Function Reference (module by module).....	11
	4.1 board.c.....	12
	4.2 players.c.....	17
	4.3 game.c.....	31
	4.4 finance.c.....	47
	4.5 bank.c.....	79
	4.6 auction.c.....	90
	4.7 events.c.....	95
	4.8 economy.c.....	104
	4.9 insurance.c.....	119
	4.10 market.c.....	131
	4.11 main.c.....	143

1 How to Use This Reference

This document describes ALL the code in the MONOPOLY.LK project: every source file, every function (signature, parameters, return value, local variables and step-by-step logic), every compile-time constant, every enum value and every struct field. Read it alongside the source files to prepare for a viva: each section below is grouped by module exactly as the code is organised. For every function and every source file, the ACTUAL C code is printed directly underneath its explanation, so you can compare what each function does with exactly how it does it.

Round cadence (from game.c playGame): processLoans, insurance expiry, ageing and modifier ticks run every round; disasters/inflation/market review every 10; economic events + regional cards every 15; government regulations every 20.

Files covered

- types.h - all #define constants, enums and the data structures (Loan, Property, Square, Player, ActiveModifier, Economy, GameState).
- functions.h - the project API: every function prototype grouped by module.
- board.c - the 40-square Sri Lanka board.
- players.c - the 4 AI strategies and all decision functions.
- game.c - the game engine: turn loop, dice, movement, jail, rounds, winner.
- finance.c - money, rent, buying, mortgages, taxes, buildings, net worth, Anti-Speculation Act.
- bank.c - loans, collateral, foreclosure.
- auction.c - the auction engine.
- events.c - event cards, economic events, regulations.
- economy.c - inflation, modifiers, ageing, maintenance, value maths.
- insurance.c - insurance policies, disasters, repairs.
- market.c - boom/decline reviews, regional cards, market conditions.
- main.c - the entry point.

2 Compile-Time Constants (types.h #define)

These constants drive every rule in the game. They are grouped by the rule area they belong to. Knowing them shows you understand the whole rulebook, since each one maps to a specific rule-LK or Section number.

CORE CONSTANTS

Macro	Value	Purpose
MAX_PLAYERS	4	Number of players in the game.
BOARD_SIZE	40	Number of squares on the board.
MAX_NAME_LENGTH	50	Buffer size for any name (player, square, group).
MAX_ROUNDS	500	Hard cap on game rounds; if reached the game ends with a net-worth comparison.
START_MONEY	30000	Each player starts with LKR 30,000.
GO_MONEY	2000	Money collected each time a player passes GO.

JAIL RULES

Macro	Value	Purpose
JAIL_BAIL	300	Cost of paying bail to leave Jail.
JAIL_SQUARE	10	Board index of the Jail square.
JAIL_MAX_TURNS	3	After this many failed turns in Jail, bail is forced.

TAXES

Macro	Value	Purpose
INCOME_TAX_RATE	15	Base Income Tax rate (%) at game start; drifts with market.
COMMUNITY_FUND_TAX_RATE	10	Base Community Development Fund rate (%) at game start.
TAX_RATE_MAX	25	Cap on the income tax rate after a regulation raises it.
HOTEL_LUXURY_TAX_PERCENT	25	Luxury tax: 25% of a hotel's value.

BUILDINGS

Macro	Value	Purpose
MAX_HOUSES	4	Maximum houses allowed on one property.
MAX_HOTELS	1	Maximum hotels allowed on one property.

BANK & LOANS

Macro	Value	Purpose
LOAN_DURATION_ROUNDS	20	Initial length of a loan in rounds.
LOAN_INTEREST_RATE	8	Starting loan interest rate (%), drifts with inflation.
LOAN_COLLATERAL_PERCENT	75	Rule-LK 2: max loan is 75% of eligible collateral.
LOAN_EXTEND_ROUNDS	10	Rule-LK 5: extending a loan adds 10 rounds.
MORTGAGE_REPAY_PERCENT	110	Redeeming a mortgage costs 110% of its value.
LOAN_INTEREST_MIN	1	Loan interest can never go below this.
LOAN_INTEREST_MAX	25	Loan interest can never exceed this.
LOAN_INTEREST_STEP	2	Cards/regulations move loan interest by 2 points.

STRATEGY THRESHOLDS

Macro	Value	Purpose
MIN_CASH_SAFETY	1000	Reserve a player keeps before spending / borrowing.
MIN_CASH_FLOOR	500	Risk Taker's floor before mortgaging.
TRADER_CASH_FLOOR	1200	Opportunistic Trader's mortgage floor.
COMFORTABLE_CASH	3000	Conservative Banker's comfort line.
BAIL_CASH_SPARE	1500	Conservative Banker's bail threshold.

RICH_CASH_LINE	5000	Rich-player threshold for certain decisions.
EXPENSIVE_PROP_PRICE	6000	Price above which the Trader insures a property.

RENT TABLES

Macro	Value	Purpose
RAILWAY_RENT_1	250	Railway rent when owning 1 station (Table 7).
RAILWAY_RENT_2	500	Railway rent when owning 2 stations.
RAILWAY_RENT_3	1000	Railway rent when owning 3 stations.
RAILWAY_RENT_4	2000	Railway rent when owning all 4 stations.
UTILITY_RENT_ONE	4	Utility rent = 4 x dice when owning 1 utility (Table 8).
UTILITY_RENT_TWO	10	Utility rent = 10 x dice when owning both utilities.

AUCTION RULES

Macro	Value	Purpose
AUCTION_OPENING_DIVISOR	2	Rule-LK 19: opening bid = asking value / 2.
AUCTION_BID_INCREMENT	250	Every bid must raise the price by this many LKR.
AUCTION_MAX_ROUNDS	200	Safety limit so bidding always terminates.

MARKET REVIEW

Macro	Value	Purpose
MARKET_REVIEW_EVERY	10	A property market review happens every 10 rounds.
MARKET_COOLDOWN_ROUNDS	30	A group can't re-trigger boom/decline for 30 rounds.

AGEING & DEPRECIATION

Macro	Value	Purpose
PROPERTY_AGE_LIMIT	50	Value starts dropping once age passes this (rule-LK 15).
DEPRECIATION_STEP	5	Depreciation ticks up every 5 rounds past the age limit.
MAX_DEPRECIATION	30	Depreciation is capped at 30%.
RENOVATION_COST_PERCENT	10	Renovation costs 10% of current value (rule-LK 17).
RENOVATION_RENT_BOOST	5	Renovated base rent rises 5%.
NEGLECT_ROUNDS	20	No maintenance for 20 rounds causes structural damage.
STRUCTURAL_VALUE_LOSS	15	Damaged building loses 15% of its value.
STRUCTURAL_RENT_LOSS	25	Damaged building loses 25% of its rent.
DAMAGE_REPAIR_MULTIPLIER	150	Fixing structural damage costs 150% of replacement value.

INSURANCE & DISASTERS

Macro	Value	Purpose
REPAIR_COST_PERCENT	30	Disaster repairs cost 30% of a property's value.
PREMIUM_BASIC_PERCENT	5	Basic insurance premium = 5% of value.
PREMIUM_COMPREHENSIVE_PERCENT	10	Comprehensive premium = 10% of value.
PREMIUM_INTERRUPTIOIN_PERCENT	15	Business Interruption premium = 15% of value.
INSURANCE_DURATION_ROUNDS	20	A policy lasts 20 rounds.
INSURANCE_RENEW_AT	10	Renew when 10 or fewer rounds remain.
INSURANCE_EXPIRY_WARNING	3	Warn 3 rounds before a policy expires.
DISASTER_WEIGHT	20	Every disaster is equally likely at base weight 20.
DISASTER_BOOST	30	Weather/unrest makes one disaster type +30 likely.
BI_LOST_INCOME_ROUNDS	5	Business Interruption: no rent for 5 rounds.

EVENT CADENCES & DECKS

Macro	Value	Purpose
EVENT_CARD_COUNT	20	Number of National Event Cards (0..19).

ECONOMIC_EVENT_COUNT	8	Number of random economic events.
REGULATION_COUNT	8	Number of government regulations.
REGIONAL_CARD_COUNT	12	Number of Regional Development Cards (Table 4).
ECONOMY_CYCLE	10	Disasters, inflation and market review every 10 rounds.
EVENT_CYCLE	15	Economic events + regional cards every 15 rounds.
REGULATION_CYCLE	20	Government regulations every 20 rounds.

ANTI-SPECULATION ACT

Macro	Value	Purpose
MAX_UNDEVELOPED_PROPS	3	Anti-Speculation Act: at most 3 undeveloped properties.
ANTI_SPEC_TRIGGER_ROUNDS	5	Forced sale after 5 consecutive rounds over the limit.

MODIFIER ECONOMY

Macro	Value	Purpose
MAX_MODIFIERS	48	Capacity of the modifiers[] array in Economy.

3 Enums and Data Structures (types.h)

3.1 enum SquareType

Value	Meaning
GO	Starting square; passing it pays GO MONEY.
PROPERTY	An ownable colour property (developable).
RAILWAY	An ownable railway; rent depends on number owned.
UTILITY	An ownable utility; rent depends on dice roll.
TAX	Income Tax square.
COMMUNITY_FUND	Community Development Fund square.
EVENT	National Event Card square.
BANK	Bank of Ceylon square (loans).
INSURANCE	Insurance square (policies).
JAIL	Jail / Just Visiting.
FREE_PARKING	Do nothing.
GO_TO_JAIL	Sent to Jail.

3.2 enum PropertyGroup

Value	Meaning
BROWN	Brown properties (Pettah, Maradana).
LIGHT_BLUE	Light Blue (Bambalapitiya, Wellawatte, Mount Lavinia).
PINK	Pink (Nugegoda, Maharagama, Kottawa).
ORANGE	Orange (Negombo, Katunayake, Ja-Ela).
RED	Red (Kandy City, Peradeniya, Katugastota).
YELLOW	Yellow (Galle Fort, Unawatuna, Hikkaduwa).
GREEN	Green (Jaffna Town, Nallur, Trincomalee).
DARK_BLUE	Dark Blue (Nuwara Eliya, Galle Face).
NO_GROUP	Sentinel = number of real groups; also used as an array bound.

3.3 enum PlayerType

Value	Meaning
AGGRESSIVE_INVESTOR	Player 1. Buys to complete monopolies.
CONSERVATIVE_BANKER	Player 2. Avoids risk and debt.
RISK_TAKER	Player 3. Spends everything; insures only after a loss.
OPPORTUNISTIC_TRADER	Player 4. Buys cheap, sells into declines.

3.4 enum InsuranceType

Value	Meaning
NO_INSURANCE	No policy.
BASIC_INSURANCE	Covers Fire and Flood; pays 80%.
COMPREHENSIVE_INSURANCE	Covers all disasters; pays 100%.
BUSINESS_INTERRUPTION	Covers all; 100% + 5 rounds of lost income.

3.5 enum DisasterType

Value	Meaning
FIRE	Fire damage.
FLOOD	Flood damage.
RIOT	Riot damage.
BUILDING_COLLAPSE	Structural collapse.

ELECTRICAL_FAILURE	Electrical failure.
--------------------	---------------------

3.6 enum ModifierType

Value	Meaning
MOD_VALUE_GLOBAL	All property values.
MOD_GROUP_VALUE	One colour group's property values.
MOD_INDEX_VALUE	A single square's value (any type).
MOD_RAIL_VALUE	All railway values.
MOD_PURCHASE_PRICE	Direct purchase prices.
MOD_RENT_GLOBAL	All rent.
MOD_GROUP_RENT	One colour group's rent.
MOD_HOTEL_RENT	Hotel rent.
MOD_RAIL_RENT	Railway rent.
MOD_UTIL_RENT	Utility rent.
MOD_CONSTRUCTION	House/hotel construction costs.
MOD_INSURANCE	Insurance premiums.
MOD_MARKET_MORTGAGE	Mortgage values.
MOD_AUCTION_PRICE	Auction starting prices.
MOD_FLOOD_RISK	Flood more likely (Heavy Monsoon).
MOD_RIOT_RISK	Riot more likely (Political Unrest).
MOD_BI_CLAIMS	BI claims inflated (Political Unrest).
MOD RECESSION	Strategy flag for Economic Recession.

3.7 enum ModifierSource

Value	Meaning
SRC_GENERAL	Events, national cards, government regulations.
SRC_BOOM	Property Market Review - booming group.
SRC_DECLINE	Property Market Review - declining group.
SRC_REGIONAL	Regional Development Card.

3.8 struct Loan

Field	Type	Purpose
active	int	1 when the player has an outstanding loan (only one loan at a time).
amount	int	Current outstanding balance in LKR (grows with compound interest).
interestRate	int	Annual/round interest rate the loan is charged at.
remainingRounds	int	Rounds left before foreclosure.

3.9 struct Property

Field	Type	Purpose
name	char[]	Property display name (echoed into the Square name too).
group	PropertyGroup	Colour group (BROWN..DARK_BLUE).
purchasePrice	int	Base price to buy (inflation and modifiers can move this).
mortgageValue	int	Value received when mortgaging (redeem costs +10%).
baseRent	int	Rent with zero houses (Table multipliers on top).
houseCost	int	Cost of building one house.
hotelCost	int	Cost of building a hotel.
owner	int	Player index who owns it; -1 means unowned.
houses	int	Number of houses (0..4).
hotel	int	1 if a hotel is built (replaces the 4 houses).
mortgaged	int	1 if the property is mortgaged (no rent until redeemed).

loanLocked	int	1 if the property is pledged as loan collateral (cannot be sold).
insurance	InsuranceType	Currently held policy type.
insuranceRoundsLeft	int	Rounds remaining on the current policy.
damaged	int	1 after a disaster until repaired (no rent).
repairCostOwed	int	Remaining repair cost after insurance compensation.
lostIncomeRoundsLeft	int	Business Interruption: rounds with no rent.
age	int	Rounds since purchase/last renovation.
depreciation	int	Percent value loss from age (0..30%).
condition	int	Building condition 0..100 (Table 3 rent mapping).
roundsSinceMaintenance	int	Rounds since last maintenance.
structurallyDamaged	int	1 when neglect (20+ rounds) caused structural damage.
preDamagePurchasePrice	int	Purchase price before structural damage (for restore).
preDamageBaseRent	int	Base rent before structural damage (for restore).
maintenanceCostMultiplierPercent	int	Cost multiplier (150% after damage, else 100%).

3.10 struct Square

Field	Type	Purpose
index	int	Board position 0..39.
type	SquareType	What this square is (GO, PROPERTY, ...).
name	char[]	Square display name.
property	Property	The embedded property data used by all square kinds.

3.11 struct Player

Field	Type	Purpose
name	char[]	Strategy name (Aggressive Investor, etc.).
strategy	PlayerType	Which strategy drives every decision.
position	int	Current board position 0..39.
cash	int	Cash balance in LKR.
inJail	int	1 while in Jail.
jailTurns	int	Consecutive jail turns while failing to roll doubles.
bankrupt	int	1 once the player is out of the game.
propertiesOwned	int	Count of colour properties owned.
railwaysOwned	int	Count of railways owned.
utilitiesOwned	int	Count of utilities owned.
loan	Loan	The player's (single) loan.
sufferedLoss	int	1 after an uninsured disaster (Risk Taker now insures).
antiSpecRounds	int	Consecutive rounds over the 3-undeveloped limit.

3.12 struct ActiveModifier

Field	Type	Purpose
type	ModifierType	What the modifier affects (value, rent, ...).
group	int	Colour group if GROUP-scoped, else -1.
index	int	Square index if INDEX-scoped, else -1.
percent	int	Multiplier percent (100 = no change, 120 = +20%).

roundsLeft	int	Rounds until the modifier expires.
source	ModifierSource	Origin used by Rule-LK 36 reporting (boom/decline/regional/general).

3.13 struct Economy

Field	Type	Purpose
inflationRate	int	Current inflation percent (-3..12) applied every 10 rounds.
loanInterestRate	int	Current economy loan interest rate (1..25).
constructionSuspendedRoundsLeft	int	Labour Strike blocks building while > 0.
closedPropertyIndex	int	Square closed by Political Rally (-1 none).
closedPropertyRoundsLeft	int	Rounds the closed property stays closed.
incomeTaxRate	int	Current income tax rate (base 15, can rise via regulation).
antiSpeculationActive	int	1 once the Anti-Speculation regulation fires.
currentCardIndex	int	Position in the National Event Card deck (0..19).
groupCooldownUntilRound	int[]	Per group: earliest round it may boom/decline again.
lastBoomGroup	int	Group boosted by the most recent market review.
lastDeclineGroup	int	Group declined by the most recent market review.
modifiers	ActiveModifier[]	Array of up to 48 active timed modifiers.
modifierCount	int	How many modifiers are currently active.

3.14 struct GameState

Field	Type	Purpose
board	Square[]	The 40 squares of the board.
players	Player[]	The 4 players.
economy	Economy	Shared economy state (inflation, modifiers, decks).

4 Function Reference (module by module)

Each module below lists its functions alphabetically in source order. For every function you get: the exact signature, a plain-English purpose, the REAL source code, the return value, the parameters, the local variables, and the step-by-step logic. Each module also starts with its file header (includes and comments).

4.1 board.c

Board construction. Creates the 40 squares of the Sri Lanka themed board in initializeBoard, using four helper setters. It is the first module called by the game.

File header (includes / comments)

```
#include <stdio.h>
#include <string.h>
#include "types.h"
#include "functions.h"
```

initializeBoard

```
void initializeBoard(GameState game[])
```

Builds the complete 40-square board: special squares, 4 railways, 2 utilities and every colour property, using the helper setters.

Actual source code

```
void initializeBoard(GameState game[])
{
    setSpecialSquare(game, 0, GO, "GO");
    setSpecialSquare(game, 2, COMMUNITY_FUND, "Community Development Fund");
    setSpecialSquare(game, 4, TAX, "Income Tax");
    setSpecialSquare(game, 7, EVENT, "National Event Card");
    setSpecialSquare(game, 10, JAIL, "Jail / Just Visiting");
    setSpecialSquare(game, 17, INSURANCE, "Sri Lanka Insurance");
    setSpecialSquare(game, 20, FREE_PARKING, "Free Parking");
    setSpecialSquare(game, 22, EVENT, "National Event Card");
    setSpecialSquare(game, 30, GO_TO_JAIL, "Go To Jail");
    setSpecialSquare(game, 33, INSURANCE, "Ceylinco Insurance");
    setSpecialSquare(game, 36, EVENT, "National Event Card");
    setSpecialSquare(game, 38, BANK, "Bank of Ceylon");

    setRailway(game, 5, "Colombo Fort Railway Station", 1500);
    setRailway(game, 15, "Kandy Railway Station", 1500);
    setRailway(game, 25, "Galle Railway Station", 1500);
    setRailway(game, 35, "Jaffna Railway Station", 1500);

    setUtility(game, 12, "Ceylon Electricity Board", 1500);
    setUtility(game, 28, "National Water Supply and Drainage Board", 1500);

    /* Brown */
    setProperty(game, 1, "Pettah", BROWN, 1500, 100, 750, 500, 2000);
    setProperty(game, 3, "Maradana", BROWN, 1800, 120, 900, 500, 2000);

    /* Light Blue */
    setProperty(game, 6, "Bambalapitiya", LIGHT_BLUE, 2500, 180, 1250, 750, 3000);
    setProperty(game, 8, "Wellawatte", LIGHT_BLUE, 2700, 200, 1350, 750, 3000);
    setProperty(game, 9, "Mount Lavinia", LIGHT_BLUE, 3000, 220, 1500, 750, 3000);

    /* Pink */
    setProperty(game, 11, "Nugegoda", PINK, 3500, 260, 1750, 1000, 4000);
    setProperty(game, 13, "Maharagama", PINK, 3800, 280, 1900, 1000, 4000);
}
```

```

setProperty(game, 14, "Kottawa", PINK, 4000, 300, 2000, 1000, 4000);

/* Orange */
setProperty(game, 16, "Negombo", ORANGE, 4500, 350, 2250, 1250, 5000);
setProperty(game, 18, "Katunayake", ORANGE, 4700, 370, 2350, 1250, 5000);
setProperty(game, 19, "Ja-Ela", ORANGE, 5000, 400, 2500, 1250, 5000);

/* Red */
setProperty(game, 21, "Kandy City", RED, 5500, 450, 2750, 1500, 6000);
setProperty(game, 23, "Peradeniya", RED, 5800, 480, 2900, 1500, 6000);
setProperty(game, 24, "Katugastota", RED, 6000, 500, 3000, 1500, 6000);

/* Yellow */
setProperty(game, 26, "Galle Fort", YELLOW, 6500, 600, 3250, 2000, 8000);
setProperty(game, 27, "Unawatuna", YELLOW, 6800, 620, 3400, 2000, 8000);
setProperty(game, 29, "Hikkaduwa", YELLOW, 7000, 650, 3500, 2000, 8000);

/* Green */
setProperty(game, 31, "Jaffna Town", GREEN, 8000, 750, 4000, 2500, 10000);
setProperty(game, 32, "Nallur", GREEN, 8300, 780, 4150, 2500, 10000);
setProperty(game, 34, "Trincomalee", GREEN, 8500, 800, 4250, 2500, 10000);

/* Dark Blue */
setProperty(game, 37, "Nuwara Eliya", DARK_BLUE, 10000, 1000, 5000, 3000, 12000);
setProperty(game, 39, "Galle Face", DARK_BLUE, 12000, 1200, 6000, 3000, 12000);
}

```

Return value

Name	Meaning
returns	void - the board is written into game[0].board.

Local variables

Name	Meaning
(none)	All data reaches the board via the setter calls.

Logic

- Lays out special squares at their fixed indices (GO=0, Income Tax=4, Jail=10, Go To Jail=30, Bank=38, etc.).
- Places 4 railways, 2 utilities, then every colour property in groups of two (Brown, Dark Blue) or three.
- Each property is given real Sri Lankan city names and prices/rents that increase with the colour group.

setProperty

```
void setProperty(GameState game[], int index, char name[], PropertyGroup group, int purchasePrice, int baseRent, int mortgageValue, int houseCost, int hotelCost)
```

Fills in one colour property square with its trade economics and resets ownership/build state.

Actual source code

```

void setProperty(GameState game[],
                int index,
                char name[],
                PropertyGroup group,
                int purchasePrice,
                int baseRent,
                int mortgageValue,
                int houseCost,
                int hotelCost)
{
    game[0].board[index].index = index;
    game[0].board[index].type = PROPERTY;
}

```

```

strcpy(game[0].board[index].name, name);
strcpy(game[0].board[index].property.name, name);

game[0].board[index].property.group = group;
game[0].board[index].property.purchasePrice = purchasePrice;
game[0].board[index].property.baseRent = baseRent;
game[0].board[index].property.mortgageValue = mortgageValue;
game[0].board[index].property.houseCost = houseCost;
game[0].board[index].property.hotelCost = hotelCost;

/* nobody owns it yet */
game[0].board[index].property.owner = -1;

game[0].board[index].property.houses = 0;
game[0].board[index].property.hotel = 0;
game[0].board[index].property.mortgaged = 0;
game[0].board[index].property.loanLocked = 0;

game[0].board[index].property.insurance = NO_INSURANCE;
game[0].board[index].property.insuranceRoundsLeft = 0;

game[0].board[index].property.damaged = 0;
game[0].board[index].property.repairCostOwed = 0;
game[0].board[index].property.lostIncomeRoundsLeft = 0;

game[0].board[index].property.age = 0;
game[0].board[index].property.depreciation = 0;
game[0].board[index].property.condition = 100;

game[0].board[index].property.roundsSinceMaintenance = 0;
game[0].board[index].property.structurallyDamaged = 0;
game[0].board[index].property.preDamagePurchasePrice = 0;
game[0].board[index].property.preDamageBaseRent = 0;
game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
index	Board position.
name	Display name.
group	Colour group of the property.
purchasePrice	Base purchase price in LKR.
baseRent	Rent with no houses.
mortgageValue	Mortgage value.
houseCost	Cost of one house.
hotelCost	Cost of a hotel.

Local variables

Name	Meaning
(none)	Sets fields directly.

Logic

- Copies the name into both the Square name and the embedded Property name.
- Zeroes owner (-1 = unowned), houses, hotel, mortgage, loan-lock, insurance, damage, age, depreciation, condition (=100), and structural-damage state so an uninitialized GameState never leaves garbage.

setRailway

```
void setRailway(GameState game[], int index, char name[], int purchasePrice)
```

Creates a railway square. Mortgage value is fixed at half the purchase price.

Actual source code

```
void setRailway(GameState game[], int index, char name[], int purchasePrice)
{
    game[0].board[index].index = index;
    game[0].board[index].type = RAILWAY;

    strcpy(game[0].board[index].name, name);
    strcpy(game[0].board[index].property.name, name);

    game[0].board[index].property.purchasePrice = purchasePrice;
    game[0].board[index].property.mortgageValue = purchasePrice / 2;
    game[0].board[index].property.owner = -1;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;

    /* railways can't have houses/hotels/insurance, but we still zero
       these out since GameState is local now and won't do it for us */
    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
    game[0].board[index].property.insuranceRoundsLeft = 0;
    game[0].board[index].property.damaged = 0;
    game[0].board[index].property.repairCostOwed = 0;
    game[0].board[index].property.lostIncomeRoundsLeft = 0;
    game[0].board[index].property.age = 0;
    game[0].board[index].property.depreciation = 0;
    game[0].board[index].property.condition = 100;
    game[0].board[index].property.roundsSinceMaintenance = 0;
    game[0].board[index].property.structurallyDamaged = 0;
    game[0].board[index].property.preDamagePurchasePrice = 0;
    game[0].board[index].property.preDamageBaseRent = 0;
    game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
    game[0].board[index].property.baseRent = 0;
    game[0].board[index].property.houseCost = 0;
    game[0].board[index].property.hotelCost = 0;
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
index	Board position.
name	Station name.
purchasePrice	Price to buy.

Local variables

Name	Meaning
(none)	

Logic

- Type is RAILWAY; baseRent/houseCost/hotelCost are zeroed (railways can't be developed).

- Condition and ageing fields are still initialized so shared property logic never reads uninitialized data.

setUtility

```
void setUtility(GameState game[], int index, char name[], int purchasePrice)
```

Creates a utility square (mortgage = half price), same zeroing discipline as railways.

Actual source code

```
void setUtility(GameState game[], int index, char name[], int purchasePrice)
{
    game[0].board[index].index = index;
    game[0].board[index].type = UTILITY;

    strcpy(game[0].board[index].name, name);
    strcpy(game[0].board[index].property.name, name);

    game[0].board[index].property.purchasePrice = purchasePrice;
    game[0].board[index].property.mortgageValue = purchasePrice / 2;
    game[0].board[index].property.owner = -1;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;

    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
    game[0].board[index].property.insuranceRoundsLeft = 0;
    game[0].board[index].property.damaged = 0;
    game[0].board[index].property.repairCostOwed = 0;
    game[0].board[index].property.lostIncomeRoundsLeft = 0;
    game[0].board[index].property.age = 0;
    game[0].board[index].property.depreciation = 0;
    game[0].board[index].property.condition = 100;
    game[0].board[index].property.roundsSinceMaintenance = 0;
    game[0].board[index].property.structurallyDamaged = 0;
    game[0].board[index].property.preDamagePurchasePrice = 0;
    game[0].board[index].property.preDamageBaseRent = 0;
    game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
    game[0].board[index].property.baseRent = 0;
    game[0].board[index].property.houseCost = 0;
    game[0].board[index].property.hotelCost = 0;
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
index	Board position.
name	Utility name.
purchasePrice	Price to buy.

Local variables

Name	Meaning
(none)	

Logic

- Type is UTILITY, rent tables use the dice roll at landing time instead of stored base rent.

setSpecialSquare

```
void setSpecialSquare(GameState game[], int index, SquareType type, char name[])
```

Creates a non-ownable square (GO, Tax, Jail, ...).

Actual source code

```
void setSpecialSquare(GameState game[],
                      int index,
                      SquareType type,
                      char name[])
{
    game[0].board[index].index = index;
    game[0].board[index].type = type;

    strcpy(game[0].board[index].name, name);

    /* Owner must be -1 (unowned), not the default 0, otherwise this
       square would look like it belongs to Player 0. */
    game[0].board[index].property.owner = -1;
    game[0].board[index].property.mortgaged = 0;
    game[0].board[index].property.loanLocked = 0;
    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
    game[0].board[index].property.insurance = NO_INSURANCE;
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
index	Board position.
type	SquareType of the special square.
name	Display name.

Local variables

Name	Meaning
(none)	

Logic

- Critical: owner must be set to -1. A zero-filled GameState would otherwise make the square look owned by Player 0.
- Only the fields that special-square logic reads are initialized.

4.2 players.c

The 4 AI strategies. Every decision a player makes funnels through a should*/wants* function that switches on the player's strategy. This is the behavioural brain of the simulation.

File header (includes / comments)

```
#include <stdio.h>
#include <string.h>
#include "types.h"
#include "functions.h"
```

initializePlayers

```
void initializePlayers(GameState game[])
```

Names the four players and gives their starting state.

Actual source code

```
void initializePlayers(GameState game[])
{
    int i;

    strcpy(game[0].players[0].name, "Aggressive Investor");
    game[0].players[0].strategy = AGGRESSIVE_INVESTOR;

    strcpy(game[0].players[1].name, "Conservative Banker");
    game[0].players[1].strategy = CONSERVATIVE_BANKER;

    strcpy(game[0].players[2].name, "Risk Taker");
    game[0].players[2].strategy = RISK_TAKER;

    strcpy(game[0].players[3].name, "Opportunistic Trader");
    game[0].players[3].strategy = OPPORTUNISTIC_TRADER;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        game[0].players[i].position = 0;
        game[0].players[i].cash = START_MONEY;

        game[0].players[i].inJail = 0;
        game[0].players[i].jailTurns = 0;

        game[0].players[i].bankrupt = 0;

        game[0].players[i].propertiesOwned = 0;
        game[0].players[i].railwaysOwned = 0;
        game[0].players[i].utilitiesOwned = 0;

        resetLoan(game, i);

        game[0].players[i].sufferedLoss = 0;
        game[0].players[i].antiSpecRounds = 0;
    }
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Loop index over MAX_PLAYERS.

Logic

- Assigns the four fixed names and strategy tags in order.
- Position 0, cash = START_MONEY, all counters zero, loan cleared via resetLoan, sufferedLoss and antiSpecRounds = 0.

shouldBuyProperty

```
int shouldBuyProperty(GameState game[], int playerIndex)
```

Decides whether the player buys the square they just landed on.

Actual source code

```
/* Decides whether a player wants to buy the property they just
   landed on. Each strategy has its own rule, from Section 3. */
int shouldBuyProperty(GameState game[], int playerIndex)
{
    int price;
    int position;

    position = game[0].players[playerIndex].position;
    price = game[0].board[position].property.purchasePrice;

    /* match the price buyProperty() actually charges (Rule-LK 31/34) */
    if(game[0].board[position].type == PROPERTY)
    {
        price = (currentMarketValue(game, position) *
                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
                                    game[0].board[position].property.group, -1)) / 100;
    }

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            /* buys to complete a monopoly group even if it dips below
               the usual 1000 reserve (Section 3.1) */
            if(wouldCompleteMonopoly(game, playerIndex, position))
                return 1;
            return (game[0].players[playerIndex].cash - price >= 1000);

        case CONSERVATIVE_BANKER:
            /* avoids property purchases entirely during a recession,
               but will still buy rail/utility income (Section 3.2) */
            if(game[0].board[position].type == PROPERTY &&
               isModifierActive(game, MOD_RECESSION, -1, -1))
                return 0;
            return (game[0].players[playerIndex].cash - price >=
                    game[0].players[playerIndex].cash / 2);

        case RISK_TAKER:
            return (game[0].players[playerIndex].cash >= price);

        case OPPORTUNISTIC_TRADER:
            /* never buys into a group that is currently declining */
            if(game[0].board[position].type == PROPERTY &&
               modifierMultiplier(game, MOD_GROUP_VALUE,
                                   game[0].board[position].property.group, -1) < 100)
                return 0;
            /* only buys cheaper properties */
            return (price <= game[0].players[playerIndex].cash * 40 / 100);

        default:
            return 0;
    }
}
```

Return value

Name	Meaning
returns	1 = buy, 0 = pass.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
price	Purchase price after market modifiers.
position	Current board square.

Logic

- Prices are matched to what buyProperty actually charges: PROPERTY uses currentMarketValue x MOD_PURCHASE_PRICE.
- Aggressive Investor: buys if it completes a monopoly, else keeps 1000 reserve.
- Conservative Banker: refuses property during recession (MOD_RECESSION active), else keeps half its cash.
- Risk Taker: buys anything it can afford.
- Opportunistic Trader: never buys into a declining group (group value < 100), only buys at <= 40% of cash.

shouldConstruct

```
int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)
```

Decides whether the player spends on a house/hotel now.

Actual source code

```
/* Decides whether a player is willing to spend on a house/hotel. */
int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (cash - cost >= MIN_CASH_SAFETY);

        case CONSERVATIVE_BANKER:
            /* never builds hotels while a loan is active - houses are
               still fine (Section 3.2) */
            if(game[0].players[playerIndex].loan.active && isHotel)
                return 0;
            return (cash - cost >= cash / 2);

        case RISK_TAKER:
            return (cash >= cost);

        case OPPORTUNISTIC_TRADER:
            /* builds eagerly while a construction subsidy is active */
            if(modifierMultiplier(game, MOD_CONSTRUCTION, -1, -1) < 100)
                return (cash - cost >= cash * 30 / 100);

            /* delays building while inflation pushes costs up */
            if(game[0].economy.inflationRate > 0)
                return (cash - cost >= cash * 60 / 100);

            return (cash - cost >= cash * 40 / 100);

        default:
            return 0;
    }
}
```

```
}
```

Return value

Name	Meaning
returns	1 = build, 0 = don't.

Parameters

Name	Meaning
playerIndex	The deciding player.
cost	Construction cost.
isHotel	1 when the building is a hotel.

Local variables

Name	Meaning
cash	Player's cash.

Logic

- Aggressive Investor: keeps MIN_CASH_SAFETY (1000) reserve.
- Conservative Banker: never builds a hotel while a loan is active; otherwise keeps half cash.
- Risk Taker: builds anything it can afford.
- Opportunistic Trader: builds eagerly when construction is subsidised (< 100%), delays while inflation pushes costs up, otherwise keeps 40%.

wantsLoan

```
int wantsLoan(GameState game[], int playerIndex)
```

Whether the player wants to take out a new loan (Bank visit, no current loan).

Actual source code

```
int wantsLoan(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return 1;

        case CONSERVATIVE_BANKER:
            /* only borrows if bankruptcy is close */
            return (cash < MIN_CASH_SAFETY);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (cash < RICH_CASH_LINE);

        default:
            return 0;
    }
}
```

Return value

Name	Meaning
------	---------

returns 1 = wants a loan, 0 = doesn't.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
cash	Player's cash.

Logic

- Aggressive Investor and Risk Taker always want a loan.
- Conservative Banker only borrows when bankrupt-close (cash < MIN_CASH_SAFETY).
- Opportunistic Trader borrows below RICH_CASH_LINE (5000).

wantsToRepayLoan

```
int wantsToRepayLoan(GameState game[], int playerIndex)
```

Whether the player repays the loan (fully) during a Bank visit.

Actual source code

```
int wantsToRepayLoan(GameState game[], int playerIndex)
{
    int cash;
    int loanAmount;

    cash = game[0].players[playerIndex].cash;
    loanAmount = game[0].players[playerIndex].loan.amount;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (cash >= 2 * loanAmount);

        case CONSERVATIVE_BANKER:
            return (cash >= loanAmount);

        case RISK_TAKER:
            /* rarely repays - prefers to keep using the cash */
            return (cash >= 5 * loanAmount);

        case OPPORTUNISTIC_TRADER:
            return (cash >= (loanAmount * 3) / 2);

        default:
            return 0;
    }
}
```

Return value

Name	Meaning
returns	1 = repay in full, 0 = keep the loan.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
------	---------

cash	Player's cash.
loanAmount	Outstanding loan balance.

Logic

- Aggressive Investor: repay once cash $\geq$ 2x loan.
- Conservative Banker: repay once cash $\geq$ loan.
- Risk Taker: rarely repays, needs cash $\geq$ 5x loan.
- Opportunistic Trader: repay once cash $\geq$ 1.5x loan.

wantsIncreaseLoan

```
int wantsIncreaseLoan(GameState game[], int playerIndex)
```

Rule-LK 5: borrow more against newly acquired collateral during a Bank visit.

Actual source code

```
/* Rule-LK 5: whether a player wants to borrow more against new
   collateral during a Bank visit. */
int wantsIncreaseLoan(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return 1;

        case CONSERVATIVE_BANKER:
            return (cash < COMFORTABLE_CASH);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (cash < RICH_CASH_LINE);

        default:
            return 0;
    }
}
```

Return value

Name	Meaning
returns	1 = increase the loan, 0 = don't.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
cash	Player's cash.

Logic

- Aggressive Investor and Risk Taker always increase.

- Conservative Banker below COMFORTABLE_CASH (3000); Opportunistic Trader below RICH_CASH_LINE (5000).

wantsExtendLoan

```
int wantsExtendLoan(GameState game[], int playerIndex)
```

Rule-LK 5: extend the loan term by 10 rounds.

Actual source code

```
/* Rule-LK 5: whether a player wants to extend the loan term. */
int wantsExtendLoan(GameState game[], int playerIndex)
{
    int remainingRounds;
    int cash;
    int loanAmount;

    remainingRounds = game[0].players[playerIndex].loan.remainingRounds;
    cash = game[0].players[playerIndex].cash;
    loanAmount = game[0].players[playerIndex].loan.amount;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            /* prefers to borrow more rather than extend */
            return 0;

        case CONSERVATIVE_BANKER:
            return (remainingRounds <= 5 && cash < loanAmount);

        case RISK_TAKER:
            return (remainingRounds <= 10);

        case OPPORTUNISTIC_TRADER:
            return (remainingRounds <= 5);

        default:
            return 0;
    }
}
```

Return value

Name	Meaning
returns	1 = extend, 0 = don't.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
remainingRounds	Rounds left on the loan.
cash	Player's cash.
loanAmount	Outstanding balance.

Logic

- Aggressive Investor prefers borrowing more over extending (never extends).
- Conservative Banker extends if <= 5 rounds left and can't afford to repay.
- Risk Taker extends when <= 10 rounds left; Opportunistic Trader when <= 5.

wantsRefinance

```
int wantsRefinance(GameState game[], int playerIndex)
```

Rule-LK 5: refinance at the economy's current (lower) rate.

Actual source code

```
/* Rule-LK 5: whether a player wants to refinance at the current rate
   (only makes sense when the current economy rate is lower). */
int wantsRefinance(GameState game[], int playerIndex)
{
    return (game[0].economy.loanInterestRate <
            game[0].players[playerIndex].loan.interestRate);
}
```

Return value

Name	Meaning
returns	1 = refinance, 0 = don't.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
(none)	Reads the economy rate and the loan's rate.

Logic

- Refinance only makes sense when the current economy rate is strictly lower than the loan's rate.

desiredInsurance

```
InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)
```

Returns the policy type the player wants for a given property.

Actual source code

```
/* Which insurance policy (if any) a player wants for a property. */
InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)
{
    int hasHotel;
    int purchasePrice;

    hasHotel = game[0].board[propIndex].property.hotel;
    purchasePrice = game[0].board[propIndex].property.purchasePrice;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            if(hasHotel)
                return COMPREHENSIVE_INSURANCE;
            return BASIC_INSURANCE;

        case CONSERVATIVE_BANKER:
            return COMPREHENSIVE_INSURANCE;

        case RISK_TAKER:
            /* only insures after already suffering a loss */
            if(!game[0].players[playerIndex].sufferedLoss)
                return NO_INSURANCE;
    }
```

```

        if(hasHotel)
            return BUSINESS_INTERRUPTION;
        return BASIC_INSURANCE;

    case OPPORTUNISTIC_TRADER:
        if(purchasePrice >= EXPENSIVE_PROP_PRICE)
            return COMPREHENSIVE_INSURANCE;
        return NO_INSURANCE;

    default:
        return NO_INSURANCE;
    }
}

```

Return value

Name	Meaning
returns	InsuranceType for the policy.

Parameters

Name	Meaning
playerIndex	The deciding player.
propIndex	The property being considered.

Local variables

Name	Meaning
hasHotel	Whether the property has a hotel.
purchasePrice	Property base purchase price.

Logic

- Aggressive Investor: Comprehensive on hotels, Basic otherwise.
- Conservative Banker: always Comprehensive.
- Risk Taker: nothing until it suffered a loss; then Business Interruption on hotels, Basic otherwise.
- Opportunistic Trader: Comprehensive only on expensive properties ($\geq$ EXPENSIVE_PROP_PRICE), else none.

shouldRenovateAgeDepreciation

```
int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)
```

Whether the player pays to renovate away age depreciation.

Actual source code

```

/* Whether a player will pay to renovate away age depreciation. */
int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)
{
    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (depreciationPercent >= 10);

        case OPPORTUNISTIC_TRADER:
            return (depreciationPercent >= 15);

        default:
            /* not specified for these two strategies - reasonable default */
            return (depreciationPercent >= 20);
    }
}

```

Return value

Name	Meaning
returns	1 = renovate, 0 = don't.

Parameters

Name	Meaning
playerIndex	The deciding player.
depreciationPercent	Current depreciation of the property.

Local variables

Name	Meaning
(none)	Switch on strategy.

Logic

- Conservative Banker renovates at $\geq 10\%$ depreciation; Opportunistic Trader at $\geq 15\%$; everyone else at $\geq 20\%$.

shouldMaintain

```
int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
```

Whether the player performs maintenance on a building.

Actual source code

```
int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    if(game[0].players[playerIndex].strategy == RISK_TAKER)
    {
        /* ignores depreciation until repair becomes unavoidable */
        return (condition < 25 && cash >= cost);
    }

    return (cash - cost >= cash * 20 / 100);
}
```

Return value

Name	Meaning
returns	1 = maintain, 0 = don't.

Parameters

Name	Meaning
playerIndex	The deciding player.
condition	Current building condition.
cost	Maintenance cost.

Local variables

Name	Meaning
cash	Player's cash.

Logic

- Risk Taker only fixes when condition < 25 (unavoidable).
- Everyone else maintains when the cost leaves $\geq 20\%$ of cash free.

willingToBid

```
int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
```

Whether the player raises the bid at an auction.

Actual source code

```
int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
{
    int marketValue;

    if(game[0].players[playerIndex].cash < candidateBid)
        return 0;

    marketValue = currentMarketValue(game, propIndex);

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (candidateBid <= (marketValue * 120) / 100);

        case CONSERVATIVE_BANKER:
            return (candidateBid < marketValue);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (candidateBid <= (marketValue * 80) / 100);

        default:
            return 0;
    }
}
```

Return value

Name	Meaning
returns	1 = bid, 0 = withdraw from this auction for good.

Parameters

Name	Meaning
playerIndex	The deciding player.
propIndex	Property being auctioned.
candidateBid	Next bid amount.

Local variables

Name	Meaning
marketValue	Current market value of the property.

Logic

- Must be able to afford the bid (cash >= candidateBid).
- Aggressive Investor: bid up to 120% of market value.
- Conservative Banker: only below market value.
- Risk Taker: bids anything.
- Opportunistic Trader: only up to 80% of market value.

shouldMortgage

```
int shouldMortgage(GameState game[], int playerIndex)
```

Whether the player mortgages a property.

Actual source code

```

int shouldMortgage(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (cash < COMFORTABLE_CASH);

        case RISK_TAKER:
            return (cash < MIN_CASH_FLOOR);

        case AGGRESSIVE_INVESTOR:
            return (cash < MIN_CASH_SAFETY);

        case OPPORTUNISTIC_TRADER:
            return (cash < TRADER_CASH_FLOOR);

        default:
            return 0;
    }
}

```

Return value

Name	Meaning
returns	1 = mortgage, 0 = don't.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
cash	Player's cash.

Logic

- Conservative Banker below COMFORTABLE_CASH; Risk Taker below MIN_CASH_FLOOR; Aggressive Investor below MIN_CASH_SAFETY; Opportunistic Trader below TRADER_CASH_FLOOR.

shouldRedeemMortgage

```
int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)
```

Whether the player redeems a mortgaged property.

Actual source code

```

int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)
{
    return (game[0].players[playerIndex].cash - redeemCost >= redeemCost);
}

```

Return value

Name	Meaning
returns	1 = redeem, 0 = don't.

Parameters

Name	Meaning
------	---------

playerIndex	The deciding player.
redeemCost	110% of the mortgage value.

Local variables

Name	Meaning
(none)	Uses redeemCost parameter.

Logic

- Redeems only when it can afford to pay the cost and still keep at least the same amount (cash - cost >= cost).

shouldPayBail

```
int shouldPayBail(GameState game[], int playerIndex)
```

Whether the player pays bail after failing to roll doubles.

Actual source code

```
/* Whether a player pays bail after failing to roll doubles.
   The assignment doesn't specify which strategy prefers which
   option, so this is a reasonable assumption per strategy. */
int shouldPayBail(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (cash >= BAIL_CASH_SPARE);

        case RISK_TAKER:
            return 0;

        case AGGRESSIVE_INVESTOR:
        case OPPORTUNISTIC_TRADER:
            return (cash >= RICH_CASH_LINE);

        default:
            return 0;
    }
}
```

Return value

Name	Meaning
returns	1 = pay bail, 0 = stay in Jail and keep rolling.

Parameters

Name	Meaning
playerIndex	The deciding player.

Local variables

Name	Meaning
cash	Player's cash.

Logic

- Conservative Banker pays when cash >= BAIL_CASH_SPARE (1500).
- Risk Taker never pays (prefers to roll).

- Aggressive Investor / Opportunistic Trader pay when cash >= RICH_CASH_LINE (5000).

4.3 game.c

The game engine: turn loop, dice, movement, Jail, round bookkeeping, turn-order resolution and the winner. playGame holds the master loop, playTurn resolves one player's landing.

File header (includes / comments)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include "types.h"
#include "functions.h"
```

startGame

```
void startGame(GameState game[])
```

Entry point driven by main.c: seeds RNG, initialises everything, runs the game and prints results.

Actual source code

```
void startGame(GameState game[])
{
    int turnOrder[MAX_PLAYERS];
    char moneyBuffer[32];

    srand((unsigned)time(NULL));

    initializeBoard(game);
    initializePlayers(game);
    initEconomy(game);
    initMarket(game);

    printf("Player 1 : %s\n", game[0].players[0].name);
    printf("Player 2 : %s\n", game[0].players[1].name);
    printf("Player 3 : %s\n", game[0].players[2].name);
    printf("Player 4 : %s\n", game[0].players[3].name);

    formatLKR(START_MONEY, moneyBuffer);
    printf("\nEach player begins with LKR %s.\n", moneyBuffer);

    determineTurnOrder(game, turnOrder);

    playGame(game, turnOrder);

    displayFinalResults(game);
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
turnOrder	Resulting order of play.
moneyBuffer	Formatted START_MONEY.

Logic

- Seeds strand with time(NULL), then calls initializeBoard, initializePlayers, initEconomy and initMarket.
- Determines turn order, calls playGame, then displays final results.

playGame

```
void playGame(GameState game[], int turnOrder[])
```

Master loop. A round completes only when EVERY solvent player has passed GO at least once (not after one turn each).

Actual source code

```
/* A round is not "each player took one turn" - it only finishes
   once every player still in the game has passed GO at least once.
   Since players move different distances each turn, this usually
   takes several turns per player, not one - so this loop keeps
   cycling through turnOrder until everyone has crossed GO. */
void playGame(GameState game[], int turnOrder[])
{
    int round;
    int turnPointer;
    int playerIndex;
    int passedGoThisRound[MAX_PLAYERS];
    int i;
    int allPassed;
    int passedGo;

    round = 1;
    turnPointer = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
        passedGoThisRound[i] = 0;

    printf("\n=====\\n");
    printf("ROUND %d\\n", round);
    printf("=====\\n");

    while(round <= MAX_ROUNDS)
    {
        playerIndex = turnOrder[turnPointer];

        if(!game[0].players[playerIndex].bankrupt)
        {
            passedGo = playTurn(game, playerIndex);

            if(passedGo)
                passedGoThisRound[playerIndex] = 1;
        }

        turnPointer = (turnPointer + 1) % MAX_PLAYERS;

        /* has every player still playing passed GO this round? */
        allPassed = 1;

        for(i = 0; i < MAX_PLAYERS; i++)
        {
            if(game[0].players[i].bankrupt)
                continue;

            if(!passedGoThisRound[i])
            {
                allPassed = 0;
            }
        }
    }
}
```

```

        break;
    }
}

if(!allPassed)
    continue;

processLoans(game);
processInsuranceExpiry(game);
ageProperties(game);
ageBuildings(game);
decrementEventTimers(game);
decrementModifiers(game);
enforceAntiSpeculation(game);

if(round % ECONOMY_CYCLE == 0)
{
    triggerDisaster(game);
    applyInflation(game);
    reviewPropertyMarket(game, round);
}

if(round % EVENT_CYCLE == 0)
{
    triggerEconomicEvent(game);
    drawRegionalCard(game);
}

if(round % REGULATION_CYCLE == 0)
{
    triggerGovernmentRegulation(game);
}

displayRoundSummary(game, round);
displayMarketConditions(game);

if(countSolventPlayers(game) <= 1)
{
    printf("\nOnly one solvent player remains. Ending game.\n");
    break;
}

round++;

if(round <= MAX_ROUNDS)
{
    printf("\n=====\\n");
    printf("ROUND %d\\n", round);
    printf("=====\\n");
}

for(i = 0; i < MAX_PLAYERS; i++)
    passedGoThisRound[i] = 0;
}
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
------	---------

turnOrder Player order fixed by the opening rolls.

Local variables

Name	Meaning
round	Current round 1..MAX_ROUNDS.
turnPointer	Cycles through turnOrder.
playerIndex	Current player.
passedGoThisRound	Per-player flag that GO was crossed this round.
i	Loop index.
allPassed	Whether every solvent player crossed GO.
passedGo	Return of playTurn for the current player.

Logic

- While round <= 500: the current player takes a turn; if they passed GO, mark it.
- Once all solvent players have passed GO, run the round-end batch: processLoans, processInsuranceExpiry, ageProperties, ageBuildings, decrementEventTimers, decrementModifiers, enforceAntiSpeculation.
- Cycle-based events: every 10 rounds triggerDisaster + applyInflation + reviewPropertyMarket; every 15 triggerEconomicEvent + drawRegionalCard; every 20 triggerGovernmentRegulation.
- Display round summary + market conditions; end early when only one solvent player remains.

playTurn

```
int playTurn(GameState game[], int playerIndex)
```

One whole turn for one player (Rule 3): repair/maintain, handle jail, roll, move, resolve landing, build, sell declining, mortgage.

Actual source code

```
/* One player's whole turn, matching Rule 3. Returns 1 if GO was
   passed at any point during the turn. */
int playTurn(GameState game[], int playerIndex)
{
    int dice;
    int pos;
    int passedGo;
    int jailResult;
    SquareType squareType;

    if(game[0].players[playerIndex].bankrupt)
        return 0;

    printf("\n-----\n");
    printf("%s's Turn\n", game[0].players[playerIndex].name);

    /* Step 1: fix any damaged, worn, or neglected buildings first */
    tryAutoRepair(game, playerIndex);
    performMaintenance(game, playerIndex);
    renovateStructuralDamage(game, playerIndex);

    passedGo = 0;

    jailResult = handleJail(game, playerIndex);

    if(jailResult == 1)
        return 0; /* was in jail - released or not, no move this turn */

    /* jailResult == 0: wasn't in jail - take a completely normal turn. */
    dice = rollDice();
    printf("%s rolled %d.\n", game[0].players[playerIndex].name, dice);
```

```

if(movePlayer(game, playerIndex, dice))
    passedGo = 1;

/* Step 4: resolve landing action */
pos = game[0].players[playerIndex].position;
squareType = game[0].board[pos].type;

if(squareType == PROPERTY || squareType == RAILWAY || squareType == UTILITY)
{
    payRent(game, playerIndex, dice);
    buyProperty(game, playerIndex);
}
else if(squareType == EVENT)
{
    executeEvent(game, playerIndex);
}
else if(squareType == TAX)
{
    payTax(game, playerIndex);
}
else if(squareType == COMMUNITY_FUND)
{
    payCommunityFundTax(game, playerIndex);
}
else if(squareType == GO_TO_JAIL)
{
    printf("%s is sent to Jail!\n", game[0].players[playerIndex].name);
    game[0].players[playerIndex].position = JAIL_SQUARE;
    game[0].players[playerIndex].inJail = 1;
    game[0].players[playerIndex].jailTurns = 0;
}
else if(squareType == BANK)
{
    handleBankVisit(game, playerIndex);
}
else if(squareType == INSURANCE)
{
    handleInsuranceVisit(game, playerIndex);
}
/* GO, JAIL (just visiting), FREE_PARKING need no action */

/* Step 6: build if eligible */
constructBuildings(game, playerIndex);

/* Section 3.4: the Opportunistic Trader first dumps any property
   that economic events have marked for a decline. */
sellDecliningProperties(game, playerIndex);

/* Step 7: mortgage or redeem if the player's situation calls for it */
handleMortgageDecisions(game, playerIndex);

return passedGo;
}

```

Return value

Name	Meaning
returns	1 if the player passed GO during the turn, else 0.

Parameters

Name	Meaning
------	---------

playerIndex The active player.

Local variables

Name	Meaning
dice	Dice roll 2..12.
pos	Position after moving.
passedGo	GO-crossed flag.
jailResult	Return of handleJail.
squareType	Type of the square landed on.

Logic

- Bankrupt players skip the turn entirely.
- Step 1: tryAutoRepair, performMaintenance, renovateStructuralDamage fix buildings first.
- If in Jail: handleJail returns 1 and no move happens this turn.
- Else roll dice, movePlayer (collects GO money), then resolve landing by square type (property/railway/util -> payRent+buyProperty, EVENT, TAX, COMMUNITY_FUND, GO_TO_JAIL, BANK, INSURANCE).
- Step 6: constructBuildings; then sellDecliningProperties; then handleMortgageDecisions.

movePlayer

```
int movePlayer(GameState game[], int playerIndex, int dice)
```

Advances the player; collects GO money if they passed or landed on GO.

Actual source code

```
/* Moves a player, returns 1 if they passed or landed on GO. */
int movePlayer(GameState game[], int playerIndex, int dice)
{
    int oldPosition;
    int passedGo;
    char moneyBuffer[32];

    oldPosition = game[0].players[playerIndex].position;
    game[0].players[playerIndex].position = (oldPosition + dice) % BOARD_SIZE;

    printf("%s moves from Square %d to Square %d.\n",
           game[0].players[playerIndex].name,
           oldPosition,
           game[0].players[playerIndex].position);

    passedGo = 0;

    if(oldPosition + dice >= BOARD_SIZE)
    {
        passedGo = 1;

        receiveMoney(game, playerIndex, GO_MONEY);

        printf("%s passed GO.\n", game[0].players[playerIndex].name);

        formatLKR(GO_MONEY, moneyBuffer);

        printf("Collected LKR %s\n", moneyBuffer);

        printf("Current Balance : LKR %d\n",
               game[0].players[playerIndex].cash);
    }

    return passedGo;
}
```

```
}
```

Return value

Name	Meaning
returns	1 if the player passed GO, else 0.

Parameters

Name	Meaning
playerIndex	The moving player.
dice	Movement amount.

Local variables

Name	Meaning
oldPosition	Position before moving.
passedGo	Flag.
moneyBuffer	Formatted GO_MONEY.

Logic

- position = (oldPosition + dice) % BOARD_SIZE.
- If old + dice >= BOARD_SIZE the player passed GO: receive GO_MONEY and print the collection.

rollDice

```
int rollDice(void)
```

Rolls two six-sided dice and returns the sum.

Actual source code

```
int rollDice(void)
{
    int die1 = rand() % 6 + 1;
    int die2 = rand() % 6 + 1;

    return die1 + die2;
}
```

Return value

Name	Meaning
returns	Sum 2..12.

Local variables

Name	Meaning
die1	First die 1..6.
die2	Second die 1..6.

Logic

- Uses rand() % 6 + 1 for each die (srand was seeded once at startGame).

handleJail

```
int handleJail(GameState game[], int playerIndex)
```

Resolves a Jail turn. Returns 1 when the player is in jail (their turn ends), 0 when they take a normal turn.

Actual source code

```

/* Handles a player's turn while they're in jail.
Return codes:
    0 = wasn't in jail - take a normal turn
    1 = was in jail (released or not) - turn ends, no move this turn */
int handleJail(GameState game[], int playerIndex)
{
    int die1, die2;

    if(!game[0].players[playerIndex].inJail)
        return 0;

    die1 = rand() % 6 + 1;
    die2 = rand() % 6 + 1;

    printf("%s is in Jail. Rolled %d and %d.\n",
        game[0].players[playerIndex].name, die1, die2);

    if(die1 == die2)
    {
        printf("Doubles! %s is released from Jail.\n",
            game[0].players[playerIndex].name);

        /* Rule 13: rolling doubles releases the player, but they do
        not move this turn - they remain on the Jail square and
        move on their next turn. */
        game[0].players[playerIndex].inJail = 0;
        game[0].players[playerIndex].jailTurns = 0;

        return 1;
    }

    /* Rule 13: paying bail is a strategy choice made after a failed
    doubles roll. The player is released but does not move this
    turn. */
    if(shouldPayBail(game, playerIndex) &&
        game[0].players[playerIndex].cash >= JAIL_BAIL)
    {
        printf("%s pays bail of LKR %d and is released from Jail.\n",
            game[0].players[playerIndex].name, JAIL_BAIL);

        payMoney(game, playerIndex, JAIL_BAIL);

        game[0].players[playerIndex].inJail = 0;
        game[0].players[playerIndex].jailTurns = 0;

        return 1;
    }

    game[0].players[playerIndex].jailTurns++;

    if(game[0].players[playerIndex].jailTurns >= JAIL_MAX_TURNS)
    {
        printf("%s has been in Jail for %d turns - must pay bail of "
            "LKR %d and is released.\n",
            game[0].players[playerIndex].name, JAIL_MAX_TURNS, JAIL_BAIL);

        payMoney(game, playerIndex, JAIL_BAIL);

        game[0].players[playerIndex].inJail = 0;
        game[0].players[playerIndex].jailTurns = 0;
    }
}

```

```

        return 1;
    }

    printf("%s remains in Jail (%d/%d turns).\n",
        game[0].players[playerIndex].name,
        game[0].players[playerIndex].jailTurns,
        JAIL_MAX_TURNS);

    return 1;
}

```

Return value

Name	Meaning
returns	0 = not in jail; 1 = was in jail (released or not).

Parameters

Name	Meaning
playerIndex	The jailed (or not) player.

Local variables

Name	Meaning
die1	First die.
die2	Second die.

Logic

- Not in jail -> return 0 immediately.
- Roll doubles -> released (free), but no move this turn (Rule 13).
- Failed doubles -> strategy-driven bail if affordable (releases, no move).
- Otherwise jailTurns++ ; at JAIL_MAX_TURNS (3) bail is forced and the player is released.
- All releases keep the player ON the Jail square this turn.

resolveGroup

```
int resolveGroup(GameState game[], int groupPlayers[], int groupSize, int output[], int startIndex)
```

Resolves a group of (possibly tied) players into a strict order by rolling dice, recursing on ties.

Actual source code

```

/* Resolves one group of (possibly tied) players into a strict order.
   Writes the result into output[] starting at startIndex, and
   returns how many players it wrote. Any players still tied after
   rolling call this function again on just themselves. */
int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
    int output[], int startIndex)
{
    int rolls[MAX_PLAYERS];
    int i, j, temp;
    int runStart, runLength;
    int tiedPlayers[MAX_PLAYERS];
    int written;
    int k;

    written = 0;

    for(i = 0; i < groupSize; i++)
    {
        rolls[i] = rollDice();
        printf("%s rolls %d.\n", game[0].players[groupPlayers[i]].name, rolls[i]);
    }
}

```

```

/* sort this group by roll, highest first */
for(i = 0; i < groupSize - 1; i++)
{
    for(j = 0; j < groupSize - 1 - i; j++)
    {
        if(rolls[j] < rolls[j + 1])
        {
            temp = rolls[j];
            rolls[j] = rolls[j + 1];
            rolls[j + 1] = temp;

            temp = groupPlayers[j];
            groupPlayers[j] = groupPlayers[j + 1];
            groupPlayers[j + 1] = temp;
        }
    }
}

/* walk the sorted list - a run of equal rolls is a tie */
i = 0;

while(i < groupSize)
{
    runStart = i;
    runLength = 1;

    while(runStart + runLength < groupSize &&
        rolls[runStart + runLength] == rolls[runStart])
    {
        runLength++;
    }

    if(runLength == 1)
    {
        output[startIndex + written] = groupPlayers[runStart];
        written++;
    }
    else
    {
        printf("Tie detected between %d players at %d. Rolling again.\n",
            runLength, rolls[runStart]);

        for(k = 0; k < runLength; k++)
            tiedPlayers[k] = groupPlayers[runStart + k];

        written += resolveGroup(game, tiedPlayers, runLength,
            output, startIndex + written);
    }

    i = runStart + runLength;
}

return written;
}

```

Return value

Name	Meaning
returns	How many players were written.

Parameters

Name	Meaning
------	---------

groupPlayers	Indices of players to order.
groupSize	How many are in this group.
output	Order filled in from startIndex.
startIndex	Where to write into output.

Local variables

Name	Meaning
rolls	Each tied player's roll.
i, j, temp	Sorting indices / swap.
runStart, runLength	Start and length of an equal-roll run.
tiedPlayers	Re-rolled subset on a tie.
written	Count written so far.
k	Copies the tied group.

Logic

- Every player rolls once; sort descending by roll (bubble sort).
- Walk the sorted list: a single roll is placed directly; an equal run recurses on just those players until all ties are broken.

determineTurnOrder

```
void determineTurnOrder(GameState game[], int turnOrder[])
```

Finds who plays first and the full order of play.

Actual source code

```
void determineTurnOrder(GameState game[], int turnOrder[])
{
    int allPlayers[MAX_PLAYERS];
    int i;

    for(i = 0; i < MAX_PLAYERS; i++)
        allPlayers[i] = i;

    printf("\nDetermining the First Player\n");

    resolveGroup(game, allPlayers, MAX_PLAYERS, turnOrder, 0);

    printf("\n%s will begin the game.\n", game[0].players[turnOrder[0]].name);

    printf("\nTurn order:\n");
    for(i = 0; i < MAX_PLAYERS; i++)
        printf("%s\n", game[0].players[turnOrder[i]].name);
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
turnOrder	Output order array.

Local variables

Name	Meaning
allPlayers	All 4 player indices.
i	Loop index.

Logic

- Seeds allPlayers = {0,1,2,3}, calls resolveGroup on all of them, and announces who begins.

countHouses

```
int countHouses(GameState game[], int playerIndex)
```

Counts houses (not hotels) owned by a player.

Actual source code

```
int countHouses(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.owner == playerIndex &&
            !game[0].board[i].property.hotel)
        {
            count += game[0].board[i].property.houses;
        }
    }

    return count;
}
```

Return value

Name	Meaning
returns	Number of houses 0..many.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
count	Running total.

Logic

- Sums property.houses over PROPERTY squares owned by the player that don't have a hotel.

countHotels

```
int countHotels(GameState game[], int playerIndex)
```

Counts hotels owned by a player.

Actual source code

```
int countHotels(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
```

```

{
    if(game[0].board[i].type == PROPERTY &&
        game[0].board[i].property.owner == playerIndex &&
        game[0].board[i].property.hotel)
    {
        count++;
    }
}

return count;
}

```

Return value

Name	Meaning
returns	Number of hotels.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
count	Running total.

Logic

- Counts PROPERTY squares owned with property.hotel == 1.

displayRoundSummary

```
void displayRoundSummary(GameState game[], int round)
```

Prints every player's cash, net worth, properties, buildings, loan and bankrupt status.

Actual source code

```

void displayRoundSummary(GameState game[], int round)
{
    int i;

    printf("\n=====\\n");
    printf("Round %d Summary\\n", round);
    printf("=====\\n");

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        char cashBuf[32];
        char netBuf[32];
        char loanBuf[32];

        formatLKR(game[0].players[i].cash, cashBuf);
        formatLKR(calculateNetWorth(game, i), netBuf);

        printf("%s\\n", game[0].players[i].name);
        printf("Cash : LKR %s\\n", cashBuf);
        printf("Net Worth : LKR %s\\n", netBuf);
        printf("Properties : %d\\n", game[0].players[i].propertiesOwned);
        printf("Railways : %d\\n", game[0].players[i].railwaysOwned);
        printf("Utilities : %d\\n", game[0].players[i].utilitiesOwned);
        printf("Houses : %d\\n", countHouses(game, i));
        printf("Hotels : %d\\n", countHotels(game, i));
    }
}

```

```

        if(game[0].players[i].loan.active)
        {
            formatLKR(game[0].players[i].loan.amount, loanBuf);
            printf("Outstanding Loan : LKR %s\n", loanBuf);
        }
        else
        {
            printf("Outstanding Loan : None\n");
        }

        if(game[0].players[i].bankrupt)
            printf("Status : BANKRUPT\n");

        printf("-----\n");
    }
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
round	Round number.

Local variables

Name	Meaning
i	Player index.
cashBuf, netBuf, loanBuf	Formatted LKR strings.

Logic

- Uses formatLKR for cash/net worth/loan, countHouses/countHotels for the building counts.

countSolventPlayers

```
int countSolventPlayers(GameState game[])
```

Counts players who are not bankrupt.

Actual source code

```

int countSolventPlayers(GameState game[])
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(!game[0].players[i].bankrupt)
            count++;
    }

    return count;
}

```

Return value

Name	Meaning
returns	Number of solvent players.

Local variables

Name	Meaning
i	Player index.
count	Running total.

Logic

- Loops all players and counts !bankrupt.

determineWinner

```
int determineWinner(GameState game[])
```

Finds the winner: the last solvent player, or the highest net worth at round 500 (Rule 15).

Actual source code

```
/* Whoever is still solvent wins. If several are still solvent at
   round 500, the highest net worth wins (Rule 15). */
int determineWinner(GameState game[])
{
    int i;
    int winner;
    int bestNetWorth;
    int netWorth;

    winner = -1;
    bestNetWorth = 0;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
            continue;

        netWorth = calculateNetWorth(game, i);

        if(winner == -1 || netWorth > bestNetWorth)
        {
            winner = i;
            bestNetWorth = netWorth;
        }
    }

    if(winner == -1)
    {
        /* edge case: everyone bankrupt at once - compare everyone anyway */
        for(i = 0; i < MAX_PLAYERS; i++)
        {
            netWorth = calculateNetWorth(game, i);

            if(winner == -1 || netWorth > bestNetWorth)
            {
                winner = i;
                bestNetWorth = netWorth;
            }
        }
    }

    return winner;
}
```

Return value

Name	Meaning
------	---------

returns Winner's player index.

Local variables

Name	Meaning
i	Player index.
winner	Best player so far.
bestNetWorth	Best net worth so far.
netWorth	Current candidate's net worth.

Logic

- Iterates solvent players keeping the highest net worth.
- Edge case: if everyone is bankrupt at once, compares net worth across ALL players anyway.

displayFinalResults

```
void displayFinalResults(GameState game[])
```

Prints the winner's cash/property/loan/net worth and every player's final standing.

Actual source code

```
void displayFinalResults(GameState game[])
{
    int winner;
    int i;
    char cashBuf[32];
    char propBuf[32];
    char loanBuf[32];
    char netBuf[32];

    winner = determinewinner(game);

    printf("\n=====\\n");
    printf("GAME OVER\\n");
    printf("=====\\n");

    printf("Winner\\n%s\\n", game[0].players[winner].name);

    formatLKR(game[0].players[winner].cash, cashBuf);
    printf("Total Cash\\nLKR %s\\n", cashBuf);

    formatLKR(calculatePropertyValue(game, winner) +
              calculateBuildingValue(game, winner), propBuf);
    printf("Total Property Value\\nLKR %s\\n", propBuf);

    if(game[0].players[winner].loan.active)
    {
        formatLKR(game[0].players[winner].loan.amount, loanBuf);
        printf("Outstanding Loans\\nLKR %s\\n", loanBuf);
    }
    else
    {
        printf("Outstanding Loans\\nNone\\n");
    }

    formatLKR(calculateNetWorth(game, winner), netBuf);
    printf("Net Worth\\nLKR %s\\n", netBuf);
    printf("=====\\n");

    printf("\\nFinal Standings (all players)\\n");
    printf("=====\\n");
```

```

for(i = 0; i < MAX_PLAYERS; i++)
{
    printf("\n%s\n", game[0].players[i].name);
    printf("Cash : LKR %d\n", game[0].players[i].cash);
    printf("Net Worth : LKR %d\n", calculateNetWorth(game, i));
    printf("Properties : %d\n", game[0].players[i].propertiesOwned);
    printf("Railways : %d\n", game[0].players[i].railwaysOwned);
    printf("Utilities : %d\n", game[0].players[i].utilitiesOwned);
    if(game[0].players[i].bankrupt)
    {
        printf("Status : Bankrupt\n");
    }
    else
    {
        printf("Status : Active\n");
    }
}
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
winner	determineWinner result.
i	Player index.

cashBuf, propBuf, loanBuf,

netBuf Formatted values.

Logic

- Property value = calculatePropertyValue + calculateBuildingValue.

4.4 finance.c

The economic core (largest file). Ownership, money movement, rent, buying, mortgages, taxes, building (Rule 9), net worth and the Anti-Speculation Act.

File header (includes / comments)

```

#include <stdio.h>
#include "types.h"
#include "functions.h"

```

stripOwnership

```
void stripOwnership(GameState game[], int index)
```

Resets a square to unowned: no owner, mortgage, loan pledge or insurance.

Actual source code

```

/* Sets a square back to "unowned": no owner, no mortgage, no loan
   pledge and no insurance. Used when selling, foreclosing or
   liquidating a property. */
void stripOwnership(GameState game[], int index)
{
    game[0].board[index].property.owner = -1;
}

```

```

game[0].board[index].property.mortgaged = 0;
game[0].board[index].property.loanLocked = 0;
game[0].board[index].property.insurance = NO_INSURANCE;
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
index	Square to reset.

Local variables

Name	Meaning
(none)	

Logic

- Used by sales, foreclosure and bankruptcy liquidation. Does not clear buildings - callers do that with demolishBuildingsOn.

resetLoan

```
void resetLoan(GameState game[], int playerIndex)
```

Wipes a player's loan (active, amount, rate, rounds).

Actual source code

```

/* Clears a player's loan completely (amount, rate and rounds). */
void resetLoan(GameState game[], int playerIndex)
{
    game[0].players[playerIndex].loan.active = 0;
    game[0].players[playerIndex].loan.amount = 0;
    game[0].players[playerIndex].loan.interestRate = 0;
    game[0].players[playerIndex].loan.remainingRounds = 0;
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
(none)	

Logic

- Used at game start and after full repayment / foreclosure.

adjustOwnedCount

```
void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
```

Updates the player's owned counter to match the square's type.

Actual source code

```

/* Adds delta to the owned-count that matches the square's type
(property, railway or utility). Pass 1 when buying, -1 when
selling or losing a square. */
void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
{
    if(game[0].board[index].type == PROPERTY)
        game[0].players[playerIndex].propertiesOwned += delta;
    else if(game[0].board[index].type == RAILWAY)
        game[0].players[playerIndex].railwaysOwned += delta;
    else if(game[0].board[index].type == UTILITY)
        game[0].players[playerIndex].utilitiesOwned += delta;
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.
index	Square whose owner changed.
delta	+1 for a buy, -1 for a loss.

Local variables

Name	Meaning
(none)	

Logic

- PROPERTY -> propertiesOwned, RAILWAY -> railwaysOwned, UTILITY -> utilitiesOwned.

groupOf

```
PropertyGroup groupOf(GameState game[], int index)
```

Returns the colour group of a square, or -1 if it isn't a property.

Actual source code

```

/* The colour group of a square, or -1 if it isn't a property. */
PropertyGroup groupOf(GameState game[], int index)
{
    if(game[0].board[index].type == PROPERTY)
        return game[0].board[index].property.group;

    return (PropertyGroup)-1;
}

```

Return value

Name	Meaning
returns	PropertyGroup, or (PropertyGroup)-1.

Parameters

Name	Meaning
index	Square index.

Local variables

Name	Meaning
------	---------

(none)

Logic

- Used on rails/utilities too - they return -1 so group-scoped maths ignores them.

receiveMoney

```
void receiveMoney(GameState game[], int playerIndex, int amount)
```

Adds money to a player, but only if they are not bankrupt.

Actual source code

```
void receiveMoney(GameState game[], int playerIndex, int amount)
{
    /* bankrupt players take no further part in the game (Section 1.3) */
    if(game[0].players[playerIndex].bankrupt)
        return;

    game[0].players[playerIndex].cash += amount;
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	Receiver.
amount	Amount in LKR.

Local variables

Name	Meaning
(none)	

Logic

- The bankrupt check is important: bankrupt players take no further part, so GO and rent must not revive them.

payMoney

```
void payMoney(GameState game[], int playerIndex, int amount)
```

Subtracts money, escaping bankruptcy: a Risk Taker first sells its cheapest undeveloped property; if still negative the player is bankrupt and assets are liquidated.

Actual source code

```
void payMoney(GameState game[], int playerIndex, int amount)
{
    game[0].players[playerIndex].cash -= amount;

    /* Section 3.4: a Risk Taker facing bankruptcy first sells their
    cheapest undeveloped property to the Bank to raise cash.
    Loan-locked collateral (Rule-LK 3) can't be sold, so stop as
    soon as a sale fails to reduce the stock of undeveloped
    properties - otherwise this loop would never end. */
    while(game[0].players[playerIndex].cash < 0 &&
        !game[0].players[playerIndex].bankrupt &&
        game[0].players[playerIndex].strategy == RISK_TAKER)
    {
        int before = countUndevelopedProperties(game, playerIndex);

        sellLowValueProperty(game, playerIndex);
    }
}
```

```

        if(countUndevelopedProperties(game, playerIndex) >= before)
            break;
    }

    if(game[0].players[playerIndex].cash < 0 && !game[0].players[playerIndex].bankrupt)
    {
        game[0].players[playerIndex].bankrupt = 1;

        printf("\n*** BANKRUPTCY ***\n");
        printf("%s has been declared bankrupt.\n",
            game[0].players[playerIndex].name);

        liquidateBankruptAssets(game, playerIndex);

        /* cash never stays negative - assets were liquidated, so any
           remaining debt is written off (Section 1.3) */
        game[0].players[playerIndex].cash = 0;
    }
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	Payer.
amount	Amount in LKR.

Local variables

Name	Meaning
before	Count of undeveloped properties before the sale.

Logic

- Risk Taker loop: while cash < 0 and not bankrupt and strategy == RISK_TAKER, sell the cheapest undeveloped property; break if a sale didn't reduce the count (all remaining are loan-locked).
- If still negative after the loop, mark bankrupt, print the announcement, liquidate assets, then clamp cash to 0.

liquidateBankruptAssets

```
void liquidateBankruptAssets(GameState game[], int playerIndex)
```

Rule 14: on bankruptcy, demolishes buildings and auctions every owned square.

Actual source code

```

/* Rule 14: on bankruptcy, buildings are demolished and every
   property/railway/utility owned is auctioned off (Rule-LK 19). */
void liquidateBankruptAssets(GameState game[], int playerIndex)
{
    int i;

    printf("%s's remaining assets are being liquidated.\n",
        game[0].players[playerIndex].name);

    resetLoan(game, playerIndex);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)

```

```

        continue;

        demolishBuildingsOn(game, i);

        stripOwnership(game, i);
        game[0].board[i].property.damaged = 0;
        game[0].board[i].property.repairCostOwed = 0;

        adjustOwnedCount(game, playerIndex, i, -1);

        runAuction(game, i, -1);
    }
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The bankrupt player.

Local variables

Name	Meaning
i	Board index.

Logic

- Clears the loan, then for every owned square: demolishBuildingsOn, stripOwnership, clear damage, adjustOwnedCount(-1) and runAuction with no seller.

payTax

```
void payTax(GameState game[], int playerIndex)
```

Income Tax: $\text{currentTaxRatePercent}(\text{INCOME_TAX_RATE}) \times \text{net worth}$.

Actual source code

```

/* Income Tax (Rule-LK): 15% of the player's net worth at the base
   rate. Market conditions scale the rate (currentTaxRatePercent). */
void payTax(GameState game[], int playerIndex)
{
    int rate;
    int netWorth;
    int amount;

    rate = currentTaxRatePercent(game, INCOME_TAX_RATE);
    netWorth = calculateNetWorth(game, playerIndex);
    amount = (netWorth * rate) / 100;

    /* a bankrupt player's net worth can be negative - a tax can never
       pay the player money, so clamp it at zero */
    if(amount < 0)
        amount = 0;

    printf("\n%s landed on Income Tax.\n",
           game[0].players[playerIndex].name);

    printf("Tax Rate : %d%%\n", rate);
    printf("Net Worth : LKR %d\n", netWorth);
    printf("%s paid tax : LKR %d\n",

```

```

        game[0].players[playerIndex].name,
        amount);

    payMoney(game, playerIndex, amount);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The taxpayer.

Local variables

Name	Meaning
rate	Effective tax rate.
netWorth	Net worth (can be negative for a broke player).
amount	Tax owed.

Logic

- Clamps amount at 0 - a tax must never pay a player (bug fix).

payCommunityFundTax

```
void payCommunityFundTax(GameState game[], int playerIndex)
```

Community Development Fund: $\text{currentTaxRatePercent}(\text{COMMUNITY_FUND_TAX_RATE}) \times \text{property (land) value, not buildings.}$

Actual source code

```

/* Community Development Fund (Rule-LK): 10% of the player's total
   assets at current market rate. Only property (land) counts -
   buildings are not included. The 10% rate is scaled by market
   conditions just like the Income Tax rate. */
void payCommunityFundTax(GameState game[], int playerIndex)
{
    int rate;
    int assets;
    int amount;

    rate = currentTaxRatePercent(game, COMMUNITY_FUND_TAX_RATE);
    assets = calculatePropertyValue(game, playerIndex);
    amount = (assets * rate) / 100;

    printf("\n%s landed on Community Development Fund.\n",
        game[0].players[playerIndex].name);

    printf("Tax Rate : %d%%\n", rate);
    printf("Property Assets (market rate) : LKR %d\n", assets);
    printf("%s paid community development tax : LKR %d\n",
        game[0].players[playerIndex].name,
        amount);

    payMoney(game, playerIndex, amount);
}

```

Return value

Name	Meaning
------	---------

returns void.

Parameters

Name	Meaning
playerIndex	The taxpayer.

Local variables

Name	Meaning
rate	Effective rate.
assets	calculatePropertyValue result.
amount	Tax owed.

Logic

- Only land counts: assets come from calculatePropertyValue which excludes buildings.

findPropertyToMortgage

```
int findPropertyToMortgage(GameState game[], int playerIndex)
```

Finds the first mortgageable owned property.

Actual source code

```
int findPropertyToMortgage(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        /* must be undeveloped before it can be mortgaged */
        if(game[0].board[i].type == PROPERTY &&
            (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
        {
            continue;
        }

        return i;
    }

    return -1;
}
```

Return value

Name	Meaning
returns	Square index, or -1 if none.

Local variables

Name	Meaning
i	Board index.

Logic

- Skips non-owned, already mortgaged, loan-locked, and developed (houses/hotel) properties.

findMortgagedProperty

```
int findMortgagedProperty(GameState game[], int playerIndex)
```

Finds the first mortgaged owned property.

Actual source code

```
int findMortgagedProperty(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner == playerIndex &&
            game[0].board[i].property.mortgaged)
        {
            return i;
        }
    }

    return -1;
}
```

Return value

Name	Meaning
returns	Square index, or -1 if none.

Local variables

Name	Meaning
i	Board index.

Logic

- Checks owner and mortgaged flag.

mortgageProperty

```
void mortgageProperty(GameState game[], int playerIndex)
```

Mortgages the first eligible property if the strategy wants to.

Actual source code

```
void mortgageProperty(GameState game[], int playerIndex)
{
    int propIndex;
    int payout;
    PropertyGroup group;

    if(!shouldMortgage(game, playerIndex))
        return;

    propIndex = findPropertyToMortgage(game, playerIndex);

    if(propIndex == -1)
        return;

    /* Rule-LK 34: a booming group's mortgage values rise by 15%, a
       declining group's fall by 10%. Railways and utilities are
       unaffected. */
    group = groupOf(game, propIndex);
```

```

    payout = (game[0].board[propIndex].property.mortgageValue *
              modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1)) / 100;

    game[0].board[propIndex].property.mortgaged = 1;

    receiveMoney(game, playerIndex, payout);

    printf("\n%s mortgaged %s for LKR %d.\n",
          game[0].players[playerIndex].name,
          game[0].board[propIndex].name,
          payout);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
propIndex	Chosen property.
payout	Mortgage value after market modifier.
group	The property's group for MOD_MARKET_MORTGAGE.

Logic

- Payout = mortgageValue x modifierMultiplier(MOD_MARKET_MORTGAGE, group) - booming groups +15%, declining -10% (Rule-LK 34).
- Sets mortgaged = 1 and receives the payout.

redeemMortgage

```
void redeemMortgage(GameState game[], int playerIndex)
```

Redeems the first mortgaged property at 110% of value.

Actual source code

```

void redeemMortgage(GameState game[], int playerIndex)
{
    int propIndex;
    int redeemCost;

    propIndex = findMortgagedProperty(game, playerIndex);

    if(propIndex == -1)
        return;

    /* pay back the mortgage value plus 10% interest */
    redeemCost = (game[0].board[propIndex].property.mortgageValue * MORTGAGE_REPAY_PERCENT) / 100;

    if(!shouldRedeemMortgage(game, playerIndex, redeemCost))
        return;

    if(game[0].players[playerIndex].cash < redeemCost)
        return;

    payMoney(game, playerIndex, redeemCost);
}

```

```

game[0].board[propIndex].property.mortgaged = 0;

printf("\n%s redeemed the mortgage on %s for LKR %d.\n",
    game[0].players[playerIndex].name,
    game[0].board[propIndex].name,
    redeemCost);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
propIndex	Chosen mortgaged property.
redeemCost	110% of mortgage value.

Logic

- Only acts if the strategy wants to and the player can afford the cost.

handleMortgageDecisions

```
void handleMortgageDecisions(GameState game[], int playerIndex)
```

The turn-end mortgage step: try redeeming first, then mortgaging.

Actual source code

```

void handleMortgageDecisions(GameState game[], int playerIndex)
{
    redeemMortgage(game, playerIndex);
    mortgageProperty(game, playerIndex);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
(none)	

Logic

- Redeem is attempted before mortgage so a player never mortgages and immediately redeems.

calculateRent

```
int calculateRent(GameState game[], int position)
```

Rent for a normal property from Table 6 (houses/hotel), scaled by group/global/hotel modifiers and building condition (Table 3).

Actual source code

```

/* Rent for a normal property, based on houses/hotel (Table 6),
   scaled by any active bonuses and by building condition (Table 3). */
int calculateRent(GameState game[], int position)
{
    int rent;
    int conditionPercent;
    int houses;
    int hotel;
    int baseRent;
    PropertyGroup group;
    int condition;

    houses = game[0].board[position].property.houses;
    hotel = game[0].board[position].property.hotel;
    baseRent = game[0].board[position].property.baseRent;
    group = game[0].board[position].property.group;
    condition = game[0].board[position].property.condition;

    if(hotel)
    {
        rent = baseRent * 10;
    }
    else
    {
        if(houses == 0)
            rent = baseRent * 1;
        else if(houses == 1)
            rent = baseRent * 2;
        else if(houses == 2)
            rent = baseRent * 3;
        else if(houses == 3)
            rent = baseRent * 5;
        else
            rent = baseRent * 7;
    }

    /* Rule-LK 34: all matching modifiers multiply together. */
    rent = (rent * modifierMultiplier(game, MOD_GROUP_RENT, group, -1)) / 100;
    rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    if(hotel)
        rent = (rent * modifierMultiplier(game, MOD_HOTEL_RENT, -1, -1)) / 100;

    if(houses > 0 || hotel)
    {
        conditionPercent = rentConditionPercent(condition);
        rent = (rent * conditionPercent) / 100;
    }

    return rent;
}

```

Return value

Name	Meaning
returns	Rent in LKR.

Parameters

Name	Meaning
------	---------

position The square.

Local variables

Name	Meaning
rent	Running rent.
conditionPercent	Table 3 mapping.
houses	Houses on the property.
hotel	Hotel flag.
baseRent	Base rent.
group	The property's group.
condition	Building condition.

Logic

- Multipliers: 0 houses x1, x2, x3, x5, x7; hotel x10.
- Then x MOD_GROUP_RENT x MOD_RENT_GLOBAL, and x MOD_HOTEL_RENT if a hotel.
- Developed buildings also scale by rentConditionPercent(condition) - excellent 100% down to closed 0%.

calculateRailwayRent

```
int calculateRailwayRent(GameState game[], int playerIndex)
```

Railway rent from Table 7 based on how many stations the OWNER has, scaled by rail/global rent modifiers.

Actual source code

```
/* Rent for a railway, based on how many the same owner has (Table 7). */
int calculateRailwayRent(GameState game[], int playerIndex)
{
    int i;
    int count;
    int baseRent;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == RAILWAY &&
            game[0].board[i].property.owner == playerIndex)
        {
            count++;
        }
    }

    if(count == 1)
        baseRent = RAILWAY_RENT_1;
    else if(count == 2)
        baseRent = RAILWAY_RENT_2;
    else if(count == 3)
        baseRent = RAILWAY_RENT_3;
    else if(count == 4)
        baseRent = RAILWAY_RENT_4;
    else
        baseRent = 0;

    baseRent = (baseRent * modifierMultiplier(game, MOD_RAIL_RENT, -1, -1)) / 100;
    baseRent = (baseRent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    return baseRent;
}
```

Return value

Name	Meaning
returns	Rent in LKR.

Parameters

Name	Meaning
playerIndex	The railway owner.

Local variables

Name	Meaning
i	Board index.
count	Number of owned stations.
baseRent	Table lookup by count.

Logic

- Uses the owner's count (passed the owner index, not the square).

calculateUtilityRent

```
int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)
```

Utility rent from Table 8: dice x 4 (one utility) or dice x 10 (both), scaled by util/global modifiers.

Actual source code

```
/* Rent for a utility, based on the dice roll just made (Table 8). */
int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)
{
    int i;
    int count;
    int rent;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == UTILITY &&
            game[0].board[i].property.owner == playerIndex)
        {
            count++;
        }
    }

    if(count == 2)
        rent = UTILITY_RENT_TWO * diceValue;
    else if(count == 1)
        rent = UTILITY_RENT_ONE * diceValue;
    else
        rent = 0;

    rent = (rent * modifierMultiplier(game, MOD_UTIL_RENT, -1, -1)) / 100;
    rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;

    return rent;
}
```

Return value

Name	Meaning
returns	Rent in LKR.

Parameters

Name	Meaning
playerIndex	The utility owner.
diceValue	The roll that just happened.

Local variables

Name	Meaning
i	Board index.
count	Owned utilities.
rent	Running rent.

Logic

- Only uses the dice value, so it must be called from payRent with the landing dice.

groupSize

```
int groupSize(GameState game[], PropertyGroup group)
```

How many squares belong to a colour group.

Actual source code

```
int groupSize(GameState game[], PropertyGroup group)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
            count++;
    }

    return count;
}
```

Return value

Name	Meaning
returns	Count (2 or 3 in this board).

Parameters

Name	Meaning
group	The group.

Local variables

Name	Meaning
i	Board index.
count	Running total.

Logic

- Counts PROPERTY squares whose group == group.

ownsMonopoly

```
int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
```

Whether the player owns every member of the group.

Actual source code

```

int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
{
    int i;
    int owned;

    owned = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
        {
            if(game[0].board[i].property.owner == playerIndex)
                owned++;
        }
    }

    return (owned == groupSize(game, group));
}

```

Return value

Name	Meaning
returns	1 = full monopoly, 0 = not.

Parameters

Name	Meaning
playerIndex	The player.
group	The group.

Local variables

Name	Meaning
i	Board index.
owned	Count owned by the player.

Logic

- owned == groupSize(group).

developGroup

```
int developGroup(GameState game[], int playerIndex, PropertyGroup group)
```

Builds one house (or the group's hotel) keeping development even (Rule 9). Returns 1 if anything was built.

Actual source code

```

/* Builds one house (or upgrades to a hotel) somewhere in this
   colour group. Always adds to whichever property currently has the
   fewest houses, so development stays even (Rule 9). Returns 1 if
   something was actually built, 0 if not - constructBuildings()
   uses this to keep building repeatedly in the same turn. */
int developGroup(GameState game[], int playerIndex, PropertyGroup group)
{
    int i;
    int minHouses;
    int targetIndex;
    int allFourHouses;

    if(!ownsMonopoly(game, playerIndex, group))
        return 0;

    minHouses = MAX_HOUSES + 1;
    targetIndex = -1;

```

```

allFourHouses = 1;

for(i = 0; i < BOARD_SIZE; i++)
{
    if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
    {
        /* can't build on a mortgaged, damaged or loan-locked
           property - those earn no rent and/or are pledged */
        if(game[0].board[i].property.mortgaged ||
           game[0].board[i].property.damaged ||
           game[0].board[i].property.loanLocked)
        {
            /* Rule 9: a blocked member that isn't fully developed
               means the group can't support a hotel yet */
            if(!game[0].board[i].property.hotel &&
               game[0].board[i].property.houses < MAX_HOUSES)
            {
                allFourHouses = 0;
            }

            continue;
        }

        if(game[0].board[i].property.hotel)
            continue;

        if(game[0].board[i].property.houses < MAX_HOUSES)
            allFourHouses = 0;

        if(game[0].board[i].property.houses < minHouses)
        {
            minHouses = game[0].board[i].property.houses;
            targetIndex = i;
        }
    }
}

if(targetIndex == -1)
    return 0; /* every property already has a hotel */

/* Rule 9: never pile a 5th house onto one property just because a
   group member is blocked from building */
if(game[0].board[targetIndex].property.houses >= MAX_HOUSES)
    return 0;

if(allFourHouses)
{
    int hotelCost;
    char costBuf[32];

    /* Rule-LK 13/31/34: base hotel cost is permanent; temporary
       events only add modifiers on top. A market boom's +10%
       applies only to the booming group, while event/regulation
       modifiers (group -1) are global. */
    hotelCost = (game[0].board[targetIndex].property.hotelCost *
                 modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

    if(!shouldConstruct(game, playerIndex, hotelCost, 1))
        return 0;

    if(game[0].players[playerIndex].cash < hotelCost)
        return 0;
}

```

```

        payMoney(game, playerIndex, hotelCost);

        game[0].board[targetIndex].property.houses = 0;
        game[0].board[targetIndex].property.hotel = 1;

        printf("\n%s upgraded %s to a Hotel.\n",
               game[0].players[playerIndex].name,
               game[0].board[targetIndex].name);

        formatLKR(hotelCost, costBuf);
        printf("Construction Cost : LKR %s.\n", costBuf);

        return 1;
    }

    {
        int houseCost;
        char costBuf[32];

        houseCost = (game[0].board[targetIndex].property.houseCost *
                     modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;

        if(!shouldConstruct(game, playerIndex, houseCost, 0))
            return 0;

        if(game[0].players[playerIndex].cash < houseCost)
            return 0;

        payMoney(game, playerIndex, houseCost);

        game[0].board[targetIndex].property.houses++;

        printf("\n%s constructed one house on %s.\n",
               game[0].players[playerIndex].name,
               game[0].board[targetIndex].name);

        formatLKR(houseCost, costBuf);
        printf("Construction Cost : LKR %s.\n", costBuf);

        return 1;
    }
}

```

Return value

Name	Meaning
returns	1 = built, 0 = nothing built.

Parameters

Name	Meaning
playerIndex	The player.
group	The group to develop.

Local variables

Name	Meaning
i	Board index.
minHouses	Fewest houses among eligible members.
targetIndex	The property to build on.

allFourHouses	1 when every unblocked member has 4 houses (hotel allowed).
hotelCost	Modified hotel cost.
houseCost	Modified house cost.
costBuf	Formatted cost.

Logic

- Only runs on full monopolies.
- Blocked member (mortgaged/damaged/loan-locked) that isn't fully developed prevents the whole group's hotel (Rule 9).
- Always adds to the property with the fewest houses; never exceeds 4 houses on one property (Rule 9 bug fix).
- Builds a hotel when allFourHouses is true; costs are scaled by MOD_CONSTRUCTION (group-scoped so a boom's +10% only hits its own group).
- Strategy decision via shouldConstruct before paying.

constructBuildings

```
void constructBuildings(GameState game[], int playerIndex)
```

Builds the maximum affordable buildings in one turn by repeatedly calling developGroup per group.

Actual source code

```
/* Keeps building on each colour group, one step at a time, until
   either nothing more can be built there this turn (unaffordable,
   the strategy doesn't want to, or the group is fully built up) -
   this is what lets a strategy build "the maximum possible number
   of houses immediately" in a single turn instead of one per turn. */
void constructBuildings(GameState game[], int playerIndex)
{
    PropertyGroup group;
    int builtSomething;
    int safetyLimit;

    if(game[0].economy.constructionSuspendedRoundsLeft > 0)
        return;

    for(group = BROWN; group < NO_GROUP; group++)
    {
        safetyLimit = 0;

        do
        {
            builtSomething = developGroup(game, playerIndex, group);
            safetyLimit++;
        }
        while(builtSomething && safetyLimit < 20);
    }
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
------	---------

group	Iterates BROWN..DARK_BLUE.
builtSomething	Whether the last developGroup call built.
safetyLimit	Protects against an endless build loop.

Logic

- Skips everything while constructionSuspendedRoundsLeft > 0 (Labour Strike).
- Per group, keeps calling developGroup until it returns 0 or a 20-call safety limit is hit.

calculatePropertyValue

```
int calculatePropertyValue(GameState game[], int playerIndex)
```

Total current market value of owned, unmortgaged squares.

Actual source code

```
int calculatePropertyValue(GameState game[], int playerIndex)
{
    int i;
    int total;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        /* mortgaged properties contribute zero to net worth */
        if(game[0].board[i].property.mortgaged)
            continue;

        /* railways and utilities also use currentMarketValue so their
           value modifiers (rail/port/water effects) apply too */
        total += currentMarketValue(game, i);
    }

    return total;
}
```

Return value

Name	Meaning
returns	Value in LKR.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
total	Running total.

Logic

- Uses currentMarketValue for railways/utilities too so their value modifiers apply.
- Mortgaged properties contribute 0.

calculateBuildingValue

```
int calculateBuildingValue(GameState game[], int playerIndex)
```

Cash value of all houses and hotels.

Actual source code

```
int calculateBuildingValue(GameState game[], int playerIndex)
{
    int i;
    int total;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.hotel)
            total += game[0].board[i].property.hotelCost;
        else
            total += game[0].board[i].property.houses * game[0].board[i].property.houseCost;
    }

    return total;
}
```

Return value

Name	Meaning
returns	Value in LKR.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
total	Running total.

Logic

- Hotel -> hotelCost; houses -> houses x houseCost.

calculateNetWorth

```
int calculateNetWorth(GameState game[], int playerIndex)
```

Rule 15 net worth: cash + property + buildings - outstanding loan.

Actual source code

```
/* Rule 15 net worth formula: cash + property value + building value,
   minus any outstanding loan amount. */
int calculateNetWorth(GameState game[], int playerIndex)
{
    int cash;
    int propertyValue;
    int buildingValue;
    int outstandingLoans;

    cash = game[0].players[playerIndex].cash;
```

```

    propertyValue = calculatePropertyValue(game, playerIndex);
    buildingValue = calculateBuildingValue(game, playerIndex);

    outstandingLoans = 0;

    if(game[0].players[playerIndex].loan.active)
        outstandingLoans = game[0].players[playerIndex].loan.amount;

    return cash + propertyValue + buildingValue - outstandingLoans;
}

```

Return value

Name	Meaning
returns	Net worth in LKR.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
cash	Player's cash.
propertyValue	calculatePropertyValue.
buildingValue	calculateBuildingValue.
outstandingLoans	Active loan balance or 0.

Logic

- Used by taxes, round summaries, the winner and the final table.

countUndevelopedProperties

```
int countUndevelopedProperties(GameState game[], int playerIndex)
```

Counts owned colour properties with no houses and no hotel.

Actual source code

```

int countUndevelopedProperties(GameState game[], int playerIndex)
{
    int i;
    int count;

    count = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.owner == playerIndex &&
            game[0].board[i].property.houses == 0 &&
            !game[0].board[i].property.hotel)
        {
            count++;
        }
    }

    return count;
}

```

Return value

Name	Meaning
------	---------

returns Count.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
count	Running total.

Logic

- The Anti-Speculation Act's core metric.

wouldCompleteMonopoly

```
int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
```

True if buying the square at pos would complete a whole colour group.

Actual source code

```
/* True if buying the property at pos would complete the whole colour
   group for the player (Section 3.1). */
int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
{
    int i;
    PropertyGroup group;

    if(game[0].board[pos].type != PROPERTY)
        return 0;

    if(game[0].board[pos].property.owner != -1)
        return 0;

    group = game[0].board[pos].property.group;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
        {
            if(i != pos && game[0].board[i].property.owner != playerIndex)
                return 0;
        }
    }

    return 1;
}
```

Return value

Name	Meaning
returns	1 = completes a monopoly, 0 = not.

Parameters

Name	Meaning
playerIndex	The player.
pos	The square being considered.

Local variables

Name	Meaning
------	---------

i	Board index.
group	Target group.

Logic

- Contributes to the Aggressive Investor's buying rule.

sellLowValueProperty

```
void sellLowValueProperty(GameState game[], int playerIndex)
```

Sells the player's cheapest undeveloped property to the Bank at market value (Risk Taker bankruptcy tactic).

Actual source code

```
/* Sells the player's lowest-value undeveloped property back to the
   Bank at market value (Section 3.4 bankruptcy tactic). */
void sellLowValueProperty(GameState game[], int playerIndex)
{
    int i;
    int best;
    int bestValue;
    int value;

    best = -1;
    bestValue = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        /* Rule-LK 3: loan-locked collateral can never be sold */
        if(game[0].board[i].property.loanLocked)
            continue;

        value = currentMarketValue(game, i);

        if(best == -1 || value < bestValue)
        {
            best = i;
            bestValue = value;
        }
    }

    if(best == -1)
        return;

    receiveMoney(game, playerIndex, bestValue);

    stripOwnership(game, best);

    adjustOwnedCount(game, playerIndex, best, -1);

    printf("\n%s sold %s to the Bank for LKR %d.\n",
        game[0].players[playerIndex].name,
        game[0].board[best].name,
```

```
        bestValue);
    }
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
best	Cheapest property.
bestValue	Its market value.
value	Current candidate's value.

Logic

- Skips developed, loan-locked property (Rule-LK 3 - collateral can never be sold).
- Receives market value, strips ownership, adjusts count.

sellDecliningProperties

```
void sellDecliningProperties(GameState game[], int playerIndex)
```

Opportunistic Trader dumps undeveloped property in groups currently declining, under a global downturn, or with a negative index modifier.

Actual source code

```
/* Section 3.4: the Opportunistic Trader sells properties expected to
   lose value following economic events - any undeveloped property in
   a group that is currently declining, under a global downturn, or
   carrying its own negative value modifier. Loan-locked collateral
   (Rule-LK 3) is never sold. */
void sellDecliningProperties(GameState game[], int playerIndex)
{
    int i;
    int globalMult;

    if(game[0].players[playerIndex].strategy != OPPORTUNISTIC_TRADER)
        return;

    if(game[0].players[playerIndex].bankrupt)
        return;

    globalMult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        int groupMult;
        int indexMult;
        int value;

        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;
```

```

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        groupMult = modifierMultiplier(game, MOD_GROUP_VALUE,
                                      game[0].board[i].property.group, -1);
        indexMult = modifierMultiplier(game, MOD_INDEX_VALUE, -1, i);

        if(globalMult >= 100 && groupMult >= 100 && indexMult >= 100)
            continue;

        value = currentMarketValue(game, i);

        receiveMoney(game, playerIndex, value);

        stripOwnership(game, i);

        adjustOwnedCount(game, playerIndex, i, -1);

        printf("\n%s sold %s to the Bank for LKR %d "
              "(expected to lose value).\n",
              game[0].players[playerIndex].name,
              game[0].board[i].name,
              value);
    }
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
globalMult	MOD_VALUE_GLOBAL multiplier.
groupMult	This group's MOD_GROUP_VALUE.
indexMult	This square's MOD_INDEX_VALUE.
value	Sell price.

Logic

- Only for OPPORTUNISTIC_TRADER who isn't bankrupt.
- Sells when any of the three multipliers is under 100 (i.e. in decline).
- Never sells developed or loan-locked properties.

sellUndevelopedPropertyToAuction

```
void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
```

Anti-Speculation Act: releases the player's lowest-value undeveloped property and auctions it.

Actual source code

```

/* Releases the player's lowest-value undeveloped property and puts
it up for auction (Anti-Speculation Act enforcement). */
void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
{
    int i;
    int best;
    int bestValue;
    int value;

    best = -1;
    bestValue = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
            continue;

        /* Rule-LK 3: loan-locked collateral can never be auctioned */
        if(game[0].board[i].property.loanLocked)
            continue;

        value = currentMarketValue(game, i);

        if(best == -1 || value < bestValue)
        {
            best = i;
            bestValue = value;
        }
    }

    if(best == -1)
        return;

    printf("\nAnti-Speculation Act : %s must sell %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[best].name);

    stripOwnership(game, best);

    adjustOwnedCount(game, playerIndex, best, -1);

    runAuction(game, best, playerIndex);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
------	---------

i	Board index.
best	Cheapest undeveloped property.
bestValue	Its market value.
value	Candidate value.

Logic

- Skips developed and loan-locked property; the seller keeps the auction proceeds (sellerIndex = playerIndex).

forceSellExcess

```
void forceSellExcess(GameState game[], int playerIndex)
```

Sells excess undeveloped property until the player is back at the 3-property limit.

Actual source code

```
/* Sells excess undeveloped properties until the player is back at
the allowed limit (or nothing more can be sold). */
void forceSellExcess(GameState game[], int playerIndex)
{
    int excess;

    excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;

    while(excess > 0)
    {
        int before = countUndevelopedProperties(game, playerIndex);

        sellUndevelopedPropertyToAuction(game, playerIndex);

        /* nothing sellable left (e.g. every excess property is
        loan-locked) - stop to avoid a non-terminating loop */
        if(countUndevelopedProperties(game, playerIndex) >= before)
            break;

        excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
    }
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
excess	Count above the limit.
before	Count before a sale (to detect stalls).

Logic

- Breaks if a sale doesn't reduce the count (everything left is loan-locked), guaranteeing termination.

enforceAntiSpeculation

```
void enforceAntiSpeculation(GameState game[])
```

Rule-LK 8: once active, force-auctions excess undeveloped property after 5 consecutive rounds over the limit.

Actual source code

```
/* Rule-LK 8 (Anti-Speculation Act): once active, a player may keep
   at most three undeveloped properties. After 5 consecutive rounds
   above the limit the excess is force-auctioned. */
void enforceAntiSpeculation(GameState game[])
{
    int i;
    int undeveloped;

    if(!game[0].economy.antiSpeculationActive)
    {
        for(i = 0; i < MAX_PLAYERS; i++)
            game[0].players[i].antiSpecRounds = 0;

        return;
    }

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
        {
            game[0].players[i].antiSpecRounds = 0;
            continue;
        }

        undeveloped = countUndevelopedProperties(game, i);

        if(undeveloped > MAX_UNDEVELOPED_PROPS)
        {
            game[0].players[i].antiSpecRounds++;

            if(game[0].players[i].antiSpecRounds >= ANTI_SPEC_TRIGGER_ROUNDS)
            {
                forceSellExcess(game, i);
                game[0].players[i].antiSpecRounds = 0;
            }
        }
        else
        {
            game[0].players[i].antiSpecRounds = 0;
        }
    }
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Player index.
undeveloped	Player's undeveloped property count.

Logic

- If the act is off, reset every player's antiSpecRounds and return.
- Per player: over the limit -> antiSpecRounds++, and when it reaches 5 force the sale and reset; at/under the limit -> reset the counter.

buyProperty

```
void buyProperty(GameState game[], int playerIndex)
```

Handles landing on an unowned square: prices it, asks the strategy, buys or sends to auction.

Actual source code

```
void buyProperty(GameState game[], int playerIndex)
{
    int pos;
    int price;
    int wantsToBuy;
    char priceBuf[32];
    char balanceBuf[32];

    pos = game[0].players[playerIndex].position;

    if(game[0].board[pos].type != PROPERTY &&
        game[0].board[pos].type != RAILWAY &&
        game[0].board[pos].type != UTILITY)
    {
        return;
    }

    if(game[0].board[pos].property.owner != -1)
        return; /* already owned */

    price = currentMarketValue(game, pos);

    if(game[0].board[pos].type == PROPERTY)
    {
        /* Rule-LK 31/34: a booming group's direct purchase prices
           rise by 15% (group-scoped, not global). Railways and
           utilities are priced at their current market value, so
           their value modifiers (MOD_RAIL_VALUE, MOD_INDEX_VALUE)
           affect their purchase price too. */
        price = (price *
                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
                                     game[0].board[pos].property.group, -1)) / 100;
    }

    wantsToBuy = shouldBuyProperty(game, playerIndex);

    if(game[0].players[playerIndex].cash < price)
        wantsToBuy = 0;

    /* Rule 5: if they don't buy directly, it goes to auction */
    if(!wantsToBuy)
    {
        runAuction(game, pos, -1);
        return;
    }

    payMoney(game, playerIndex, price);

    game[0].board[pos].property.owner = playerIndex;

    if(game[0].board[pos].type == PROPERTY)
        game[0].players[playerIndex].propertiesOwned++;
    else if(game[0].board[pos].type == RAILWAY)
        game[0].players[playerIndex].railwaysOwned++;
    else if(game[0].board[pos].type == UTILITY)
```

```

        game[0].players[playerIndex].utilitiesOwned++;

        formatLKR(price, priceBuf);
        formatLKR(game[0].players[playerIndex].cash, balanceBuf);

        printf("\n%s purchased %s for LKR %s.\n",
            game[0].players[playerIndex].name,
            game[0].board[pos].name,
            priceBuf);

        printf("Remaining Balance : LKR %s.\n",
            balanceBuf);
    }

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
pos	Current square.
price	Price after modifiers.
wantsToBuy	Strategy decision.
priceBuf, balanceBuf	Formatted values.

Logic

- Only acts on unowned PROPERTY/RAILWAY/UTILITY.
- PROPERTY price = currentMarketValue x MOD_PURCHASE_PRICE (group-scoped, boom +15%). Rails/utilities use currentMarketValue directly.
- If the strategy declines or cash is short, the property goes to auction (Rule 5).
- On purchase: payMoney, set owner, increment the right owned counter.

payRent

```
void payRent(GameState game[], int playerIndex, int diceValue)
```

Resolves rent when landing on an owned, unmortgaged, working square.

Actual source code

```

void payRent(GameState game[], int playerIndex, int diceValue)
{
    int pos;
    int owner;
    int rent;
    char rentBuf[32];

    pos = game[0].players[playerIndex].position;

    if(game[0].board[pos].type != PROPERTY &&
        game[0].board[pos].type != RAILWAY &&
        game[0].board[pos].type != UTILITY)
    {
        return;
    }

    owner = game[0].board[pos].property.owner;

```

```

if(owner == playerIndex)
{
    /* landed on your own square - no rent, maybe renovate */
    if(game[0].board[pos].type == PROPERTY)
        tryRenovateAgeDepreciation(game, playerIndex, pos);
    return;
}

if(owner == -1)
    return;

if(game[0].board[pos].property.mortgaged)
    return;

if(pos == game[0].economy.closedPropertyIndex && game[0].economy.closedPropertyRoundsLeft > 0)
{
    printf("\n%s landed on %s, but it is closed (Political Rally) - no rent.\n",
        game[0].players[playerIndex].name, game[0].board[pos].name);
    return;
}

if(game[0].board[pos].type == PROPERTY && game[0].board[pos].property.damaged)
{
    printf("\n%s landed on %s, but it is damaged and collects no rent.\n",
        game[0].players[playerIndex].name, game[0].board[pos].name);
    return;
}

if(game[0].board[pos].type == PROPERTY &&
    game[0].board[pos].property.lostIncomeRoundsLeft > 0)
{
    printf("\n%s landed on %s, but it earns no rent right now "
        "(Business Interruption).\n",
        game[0].players[playerIndex].name, game[0].board[pos].name);
    return;
}

if(game[0].board[pos].type == PROPERTY)
    rent = calculateRent(game, pos);
else if(game[0].board[pos].type == RAILWAY)
    rent = calculateRailwayRent(game, owner);
else
    rent = calculateUtilityRent(game, owner, diceValue);

payMoney(game, playerIndex, rent);
receiveMoney(game, owner, rent);

formatLKR(rent, rentBuf);

printf("\n%s landed on %s.\n",
    game[0].players[playerIndex].name,
    game[0].board[pos].name);

printf("Rent Paid : LKR %s.\n", rentBuf);

printf("Owner : %s.\n", game[0].players[owner].name);
}

```

Return value

Name	Meaning
------	---------

returns void.

Parameters

Name	Meaning
playerIndex	The player who landed.
diceValue	The roll, for utility rent.

Local variables

Name	Meaning
pos	Current square.
owner	The square's owner.
rent	Computed rent.
rentBuf	Formatted rent.

Logic

- Landed on your own property -> no rent; tries renovation for aged property.
- No rent if unowned, mortgaged, closed by Political Rally, damaged, or in Business Interruption.
- PROPERTY/RAILWAY/UTILITY dispatch to the three rent calculators.
- payMoney(tenant) and receiveMoney(owner).

4.5 bank.c

Loans, collateral and foreclosure. Rule-LK 2 (75% collateral loan), Rule-LK 5 (Bank visit actions) and Rule-LK 6/7 (foreclosure) live here.

File header (includes / comments)

```
#include <stdio.h>
#include "types.h"
#include "functions.h"
```

groupBasePrice

```
int groupBasePrice(GameState game[], PropertyGroup group)
```

The base price of a colour group - the first property's purchase price - used ONLY for loan collateral maths.

Actual source code

```
/* Rule-LK: each colour group has a base price that is used ONLY for
loan calculations (individual purchase prices are used for
buying). It matches the first property in the group on the board,
so this reads it straight from the board instead of storing a
second copy. Railways and utilities use their own mortgage value. */
int groupBasePrice(GameState game[], PropertyGroup group)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            game[0].board[i].property.group == group)
        {
            return game[0].board[i].property.purchasePrice;
        }
    }

    return 0;
}
```

Return value

Name	Meaning
returns	Base price in LKR, 0 for no match.

Parameters

Name	Meaning
group	The group.

Local variables

Name	Meaning
i	Board index.

Logic

- Reads straight from the board (first PROPERTY in the group) instead of storing a duplicate constant.
- Rail/utility are not part of this - they use their own mortgage value.

totalEligibleCollateral

```
int totalEligibleCollateral(GameState game[], int playerIndex)
```

Sum of the player's unmortgaged, un-pledged property values (group base price for colour property, mortgage value for rail/utility), each x its market modifier.

Actual source code

```
int totalEligibleCollateral(GameState game[], int playerIndex)
{
    int i;
    int total;
    int group;
    int mult;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
            game[0].board[i].type != RAILWAY &&
            game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        /* Rule-LK 34: a booming group's collateral is worth 15% more,
           a declining group's 10% less. Colour property collateral
           is based on the group base price (not the individual
           purchase price - that is only used for buying). */
        group = groupOf(game, i);

        mult = modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1);

        if(game[0].board[i].type == PROPERTY)
```

```

        total += (groupBasePrice(game, group) * mult) / 100;
    else
        total += (game[0].board[i].property.mortgageValue * mult) / 100;
    }

    return total;
}

```

Return value

Name	Meaning
returns	Collateral value in LKR.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
total	Running total.
group	Property's group.
mult	MOD_MARKET_MORTGAGE multiplier.

Logic

- Skips non-ownable, non-owned, mortgaged and loan-locked squares.
- Booming groups' collateral +15%, declining -10% (Rule-LK 34).

calculateMaxLoan

```
int calculateMaxLoan(GameState game[], int playerIndex)
```

Rule-LK 2: the maximum loan = 75% of eligible collateral.

Actual source code

```

/* Rule-LK 2: max loan is 75% of eligible collateral. */
int calculateMaxLoan(GameState game[], int playerIndex)
{
    return (totalEligibleCollateral(game, playerIndex) * LOAN_COLLATERAL_PERCENT) / 100;
}

```

Return value

Name	Meaning
returns	Max loan in LKR.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
(none)	Delegates to totalEligibleCollateral.

Logic

- Pure formula function: collateral x LOAN_COLLATERAL_PERCENT / 100.

obtainLoan

```
void obtainLoan(GameState game[], int playerIndex)
```

Takes a new loan: locks every eligible property as collateral and pays out the max loan at the current rate.

Actual source code

```
void obtainLoan(GameState game[], int playerIndex)
{
    int i;
    int maxLoan;
    char loanBuf[32];

    if(game[0].players[playerIndex].loan.active)
        return;    /* only one active loan allowed at a time */

    maxLoan = calculateMaxLoan(game, playerIndex);

    if(maxLoan <= 0)
        return;

    game[0].players[playerIndex].loan.active = 1;
    game[0].players[playerIndex].loan.amount = maxLoan;
    game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;
    game[0].players[playerIndex].loan.remainingRounds = LOAN_DURATION_ROUNDS;

    receiveMoney(game, playerIndex, maxLoan);

    printf("\n%s obtained a secured loan.\n", game[0].players[playerIndex].name);

    formatLKR(maxLoan, loanBuf);
    printf("Loan Amount : LKR %s.\n", loanBuf);
    printf("Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
    printf("Duration : %d Rounds\n", LOAN_DURATION_ROUNDS);

    printf("Collateral :\n");

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner == playerIndex &&
           !game[0].board[i].property.mortgaged &&
           !game[0].board[i].property.loanLocked)
        {
            game[0].board[i].property.loanLocked = 1;
            printf("%s\n", game[0].board[i].name);
        }
    }
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
maxLoan	Amount borrowed.
loanBuf	Formatted amount.

Logic

- Only one active loan per player at a time.
- Locked = collateral pledge (loanLocked=1) blocks selling/mortgaging (Rule-LK 3).
- 20 rounds at the economy's current interest rate.

repayLoan

```
void repayLoan(GameState game[], int playerIndex, int amount)
```

Repays part or all of the loan; on full repayment all collateral is released.

Actual source code

```
void repayLoan(GameState game[], int playerIndex, int amount)
{
    int i;

    if(!game[0].players[playerIndex].loan.active)
        return;

    if(amount > game[0].players[playerIndex].loan.amount)
        amount = game[0].players[playerIndex].loan.amount;

    if(game[0].players[playerIndex].cash < amount)
        amount = game[0].players[playerIndex].cash;

    payMoney(game, playerIndex, amount);
    game[0].players[playerIndex].loan.amount -= amount;

    printf("\n%s repaid LKR %d towards their loan.\n",
        game[0].players[playerIndex].name, amount);

    printf("Outstanding Balance : LKR %d\n",
        game[0].players[playerIndex].loan.amount);

    if(game[0].players[playerIndex].loan.amount <= 0)
    {
        resetLoan(game, playerIndex);

        for(i = 0; i < BOARD_SIZE; i++)
        {
            if(game[0].board[i].property.owner == playerIndex)
                game[0].board[i].property.loanLocked = 0;
        }

        printf("%s has fully repaid the loan. Collateral released.\n",
            game[0].players[playerIndex].name);
    }
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
------	---------

playerIndex	The player.
amount	Amount to repay.

Local variables

Name	Meaning
i	Board index.

Logic

- Clamps the payment to the loan balance and to the player's cash.
- On a zero balance: resetLoan and clear every loanLocked flag.

handleBankVisit

```
void handleBankVisit(GameState game[], int playerIndex)
```

The Bank square: repay full / increase / extend / refinance (existing loan), or obtain a new loan (Strategy 3.5). One transaction per visit.

Actual source code

```
/* Rule-LK 5: landing on the Bank offers repaying part, repaying in
   full, or (if no loan yet) obtaining one. With an existing loan a
   player can also extend the term, refinance at the current rate, or
   borrow more against newly acquired collateral. Only one
   transaction happens per visit. */
void handleBankVisit(GameState game[], int playerIndex)
{
    printf("\n%s landed on Bank of Ceylon.\n", game[0].players[playerIndex].name);

    if(game[0].players[playerIndex].loan.active)
    {
        if(wantsToRepayLoan(game, playerIndex))
        {
            int repayAmount;

            /* every strategy's wantsToRepayLoan() only triggers once
               cash can clear the whole loan, so always repay in full */
            repayAmount = game[0].players[playerIndex].loan.amount;

            repayLoan(game, playerIndex, repayAmount);
            return;
        }

        if(wantsIncreaseLoan(game, playerIndex))
        {
            increaseLoan(game, playerIndex);
            return;
        }

        if(wantsExtendLoan(game, playerIndex))
        {
            extendLoan(game, playerIndex);
            return;
        }

        if(wantsRefinance(game, playerIndex))
        {
            refinanceLoan(game, playerIndex);
            return;
        }
    }
}
```

```

        return;
    }

    if(wantsLoan(game, playerIndex))
    {
        obtainLoan(game, playerIndex);
    }
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
repayAmount	Amount to repay (always the full loan).

Logic

- With a loan, follows the preference chain: full repay (strategy), increase, extend, refinance.
- Without a loan, obtains one if the strategy wants it.

increaseLoan

```
void increaseLoan(GameState game[], int playerIndex)
```

Rule-LK 5: borrows 75% of collateral acquired since the loan was taken out.

Actual source code

```

/* Rule-LK 5: a player with an active loan may borrow more against
   collateral they've acquired since the loan was taken out. */
void increaseLoan(GameState game[], int playerIndex)
{
    int i;
    int availableCollateral;
    int additional;

    if(!game[0].players[playerIndex].loan.active)
        return;

    availableCollateral = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;
    }
}

```

```

        if(game[0].board[i].property.loanLocked)
            continue;

        if(game[0].board[i].type == PROPERTY)
        {
            availableCollateral +=
                (groupBasePrice(game, game[0].board[i].property.group) *
                 modifierMultiplier(game, MOD_MARKET_MORTGAGE,
                                     game[0].board[i].property.group, -1)) / 100;
        }
        else
        {
            availableCollateral += game[0].board[i].property.mortgageValue;
        }
    }

    additional = (availableCollateral * LOAN_COLLATERAL_PERCENT) / 100;

    if(additional <= 0)
        return;

    game[0].players[playerIndex].loan.amount += additional;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner == playerIndex &&
           !game[0].board[i].property.mortgaged &&
           !game[0].board[i].property.loanLocked)
        {
            game[0].board[i].property.loanLocked = 1;
        }
    }

    receiveMoney(game, playerIndex, additional);

    printf("\n%s increased their loan by LKR %d.\n",
           game[0].players[playerIndex].name, additional);
    printf("New Outstanding Balance : LKR %d\n",
           game[0].players[playerIndex].loan.amount);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.

availableCollateral	Value of new eligible collateral.
additional	Extra amount borrowed.

Logic

- Only when a loan is active; finds unmortgaged, un-pledged property.
- Adds the extra amount, locks the new collateral, pays out via receiveMoney.

extendLoan

```
void extendLoan(GameState game[], int playerIndex)
```

Rule-LK 5: adds 10 rounds to the loan deadline.

Actual source code

```
/* Rule-LK 5: extend the loan term by 10 rounds. */
void extendLoan(GameState game[], int playerIndex)
{
    if(!game[0].players[playerIndex].loan.active)
        return;

    game[0].players[playerIndex].loan.remainingRounds += LOAN_EXTEND_ROUNDS;

    printf("\n%s extended their loan by %d rounds.\n",
        game[0].players[playerIndex].name, LOAN_EXTEND_ROUNDS);
    printf("New Duration : %d Rounds remaining\n",
        game[0].players[playerIndex].loan.remainingRounds);
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
(none)	

Logic

- Guarded on an active loan.

refinanceLoan

```
void refinanceLoan(GameState game[], int playerIndex)
```

Rule-LK 5: switches the loan to the economy's current rate.

Actual source code

```
/* Rule-LK 5: refinance the loan at the current economy interest rate. */
void refinanceLoan(GameState game[], int playerIndex)
{
    if(!game[0].players[playerIndex].loan.active)
        return;

    game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;

    printf("\n%s refinanced their loan at the current rate of %d%%.\n",
```

```

        game[0].players[playerIndex].name,
        game[0].economy.loanInterestRate);
    }

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
(none)	

Logic

- Only beneficial when the current economy rate is lower (strategy decides).

demolishBuildingsOn

```
void demolishBuildingsOn(GameState game[], int index)
```

Zeroes houses and hotel on a property (used on seizure and bankruptcy).

Actual source code

```

void demolishBuildingsOn(GameState game[], int index)
{
    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
index	The square.

Local variables

Name	Meaning
(none)	

Logic

- Condition and damage flags are left alone - callers manage those.

foreclose

```
void foreclose(GameState game[], int playerIndex)
```

Rule-LK 6/7: a defaulted loan seizes and auctions every pledged property; the player is bankrupt if nothing is left.

Actual source code

```

/* Rule-LK 6, 7: if a loan isn't repaid in time, pledged properties
   are seized and auctioned. If nothing is left, the player is
   declared bankrupt. */
void foreclose(GameState game[], int playerIndex)
{
    int i;

```

```

int ownsAnythingLeft;

printf("\n%s has defaulted on their loan!\n", game[0].players[playerIndex].name);

ownsAnythingLeft = 0;

for(i = 0; i < BOARD_SIZE; i++)
{
    if(game[0].board[i].property.owner != playerIndex)
        continue;

    if(!game[0].board[i].property.loanLocked)
    {
        ownsAnythingLeft = 1;
        continue;
    }

    printf("%s (pledged collateral) transferred to the Bank.\n",
        game[0].board[i].name);

    demolishBuildingsOn(game, i);

    stripOwnership(game, i);

    adjustOwnedCount(game, playerIndex, i, -1);

    runAuction(game, i, -1);
}

resetLoan(game, playerIndex);

printf("Outstanding debt cleared.\n");

if(!ownsAnythingLeft && game[0].players[playerIndex].cash <= 0)
{
    game[0].players[playerIndex].bankrupt = 1;
    printf("%s has no remaining assets and is declared BANKRUPT.\n",
        game[0].players[playerIndex].name);
}
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
ownsAnythingLeft	1 if any non-pledged property remains.

Logic

- Every loanLocked property is demolished, stripped, count-adjusted and auctioned.
- Non-pledged property keeps the player solvent.
- If nothing is left and cash <= 0, the player is bankrupt.

processLoans

```
void processLoans(GameState game[])
```

Round-end loan heartbeat: compounds interest into the principal and counts down; at 0 rounds, forecloses.

Actual source code

```
void processLoans(GameState game[])
{
    int i;
    int interest;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)
            continue;

        if(!game[0].players[i].loan.active)
            continue;

        interest = (game[0].players[i].loan.amount * game[0].players[i].loan.interestRate) / 100;
        game[0].players[i].loan.amount += interest;

        game[0].players[i].loan.remainingRounds--;

        printf("\n%s's loan accrued LKR %d interest. New balance : LKR %d\n",
            game[0].players[i].name, interest, game[0].players[i].loan.amount);

        if(game[0].players[i].loan.remainingRounds <= 0)
        {
            foreclose(game, i);
        }
    }
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Player index.
interest	Round interest amount.

Logic

- Interest = amount x rate%, written back into amount (compounding).
- remainingRounds-- , and at <= 0 foreclose.

4.6 auction.c

The auction engine (Rule-LK 19-23). Used when nobody buys directly and for forced sales (Anti-Speculation Act, bankruptcy, foreclosure).

File header (includes / comments)

```
#include <stdio.h>
#include "types.h"
#include "functions.h"
```

getAskingValue

```
int getAskingValue(GameState game[], int propIndex)
```

Rule-LK 19/32/34: asking value = current market value x auction modifier (declining groups start -25%).

Actual source code

```
/* Rule-LK 19: opening bid is 50% of market value. A market decline
   lowers the starting price by 25% (Rule-LK 32/34) - group-scoped.
   Railways and utilities use currentMarketValue too, so their value
   modifiers (Port Expansion, Water Shortage, ...) apply the same way
   they do for colour-group properties. */
int getAskingValue(GameState game[], int propIndex)
{
    int value;
    int group;

    value = currentMarketValue(game, propIndex);

    group = groupOf(game, propIndex);

    value = (value * modifierMultiplier(game, MOD_AUCTION_PRICE, group, -1)) / 100;

    return value;
}
```

Return value

Name	Meaning
returns	Asking value in LKR.

Parameters

Name	Meaning
propIndex	The square.

Local variables

Name	Meaning
value	Current market value.
group	The square's group (may be -1).

Logic

- Applies MOD_AUCTION_PRICE scoped to the group, so only the declining group's auction price drops.

runAuction

```
void runAuction(GameState game[], int propIndex, int sellerIndex)
```

Sells a property to the highest bidder among solvent players. Opening = value/2, bids rise by 250, passers are out forever. The forced seller never bids.

Actual source code

```
/* Rule-LK 19-23: runs an auction among all solvent players. A
   player who declines to bid is out permanently, so the pool of
   bidders can only shrink - guaranteeing the loop finishes.
   sellerIndex >= 0 (Anti-Speculation Act forced sale) sends the
   proceeds to that player; sellerIndex == -1 pays the Bank. */
void runAuction(GameState game[], int propIndex, int sellerIndex)
{
    int active[MAX_PLAYERS];
    int activeCount;
```

```

int i;
int currentBid;
int highBidder;
int candidateBid;
int safetyRounds;
char moneyBuf[32];

printf("\n*** AUCTION ***\n");
printf("Property : %s\n", game[0].board[propIndex].name);

currentBid = getAskingValue(game, propIndex) / AUCTION_OPENING_DIVISOR;

formatLKR(currentBid, moneyBuf);
printf("Opening Bid : LKR %s\n", moneyBuf);

highBidder = -1;
activeCount = 0;

for(i = 0; i < MAX_PLAYERS; i++)
{
    active[i] = !game[0].players[i].bankrupt;

    /* Anti-Speculation Act (Rule-LK 8): the forced seller must
       actually part with the property, so they never bid. If the
       seller were allowed to bid, they could win the auction back
       and merely pay the bid to themselves (net-zero cost), which
       defeats the forced sale. */
    if(i == sellerIndex)
        active[i] = 0;

    if(active[i])
        activeCount++;
}

safetyRounds = 0;

while(activeCount > 1 && safetyRounds < AUCTION_MAX_ROUNDS)
{
    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(!active[i])
            continue;

        if(activeCount <= 1)
            break;

        candidateBid = currentBid + AUCTION_BID_INCREMENT;

        if(willingToBid(game, i, propIndex, candidateBid))
        {
            currentBid = candidateBid;
            highBidder = i;

            formatLKR(currentBid, moneyBuf);
            printf("%s bids LKR %s.\n", game[0].players[i].name, moneyBuf);
        }
        else
        {
            active[i] = 0;
            activeCount--;

            printf("%s withdraws.\n", game[0].players[i].name);
        }
    }
}

```

```

    }
}

safetyRounds++;
}

/* If exactly one bidder is still active, give them one final
   chance to bid at the next increment before awarding. */
if(activeCount == 1 && highBidder == -1)
{
    int remaining;

    remaining = -1;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(active[i])
        {
            remaining = i;
            break;
        }
    }

    if(remaining != -1)
    {
        candidateBid = currentBid + AUCTION_BID_INCREMENT;

        if(willingToBid(game, remaining, propIndex, candidateBid))
        {
            currentBid = candidateBid;
            highBidder = remaining;

            formatLKR(currentBid, moneyBuf);
            printf("%s bids LKR %s.\n",
                  game[0].players[remaining].name, moneyBuf);
        }
    }
}

if(highBidder == -1)
{
    int lastActive;

    /* One player never got a chance to bid (e.g. the first active
       player declined while only two were in) - they may take the
       property at the opening bid. */
    lastActive = -1;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(active[i])
        {
            lastActive = i;
            break;
        }
    }

    /* No one left to bid, or they decline the opening price */
    if(lastActive == -1 || !willingToBid(game, lastActive, propIndex, currentBid))
    {
        printf("No bids received. %s remains with the Bank.\n",
              game[0].board[propIndex].name);
    }
}

```

```

        return;
    }

    highBidder = lastActive;
}

/* Always format the final winning price so the display is correct
   regardless of which code path set highBidder. */
formatLKR(currentBid, moneyBuf);

payMoney(game, highBidder, currentBid);

if(sellerIndex >= 0)
    receiveMoney(game, sellerIndex, currentBid);

game[0].board[propIndex].property.owner = highBidder;

adjustOwnedCount(game, highBidder, propIndex, 1);

printf("%s wins the auction for LKR %s.\n",
       game[0].players[highBidder].name, moneyBuf);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
propIndex	The square being auctioned.
sellerIndex	Forced seller (≥ 0) who receives the money, or -1 for the Bank.

Local variables

Name	Meaning
active	Per-player in/out flag.
activeCount	Players still bidding.
i	Loop index.
currentBid	Current highest bid.
highBidder	Leader, or -1.
candidateBid	Next proposed bid.
safetyRounds	Loop guard.
moneyBuf	Formatted bid.
remaining	Last active bidder when only one remains.
lastActive	Finder of the last active bidder.

Logic

- Opening bid = askingValue / 2.
- Bankrupt players are out; the forced seller is excluded (bug fix) so they can't 'pay themselves' (net-zero) and defeat the forced sale.
- While more than one bids: each active player either raises by 250 or withdraws permanently; capped at 200 safety rounds.
- Single remaining bidder gets one final chance; if nobody bids at all, or the last bidder declines the opening price, the property stays with the Bank.
- Winner pays, the seller (if any) receives it, ownership transfers.

4.7 events.c

Cards and regulations: 20 national event cards, 8 economic events (every 15 rounds) and 8 government regulations (every 20 rounds), plus interest clamping and timer decrementing.

File header (includes / comments)

```
#include <stdio.h>
#include <stdlib.h>
#include "types.h"
#include "functions.h"
```

clampLoanInterest

```
void clampLoanInterest(GameState game[])
```

Keeps the loan interest rate inside [1, 25].

Actual source code

```
/* Keeps the loan interest rate inside its allowed range. */
void clampLoanInterest(GameState game[])
{
    if(game[0].economy.loanInterestRate < LOAN_INTEREST_MIN)
        game[0].economy.loanInterestRate = LOAN_INTEREST_MIN;

    if(game[0].economy.loanInterestRate > LOAN_INTEREST_MAX)
        game[0].economy.loanInterestRate = LOAN_INTEREST_MAX;
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
(none)	Mutates economy.loanInterestRate.

Logic

- Called after any change to the rate.

adjustLoanInterest

```
void adjustLoanInterest(GameState game[], int deltaPercent)
```

Moves the loan interest rate by deltaPercent and clamps it.

Actual source code

```
/* Moves the loan interest rate up or down by deltaPercent. */
void adjustLoanInterest(GameState game[], int deltaPercent)
{
    game[0].economy.loanInterestRate += deltaPercent;
    clampLoanInterest(game);
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
deltaPercent	Signed change, e.g. -2 or +2.

Local variables

Name	Meaning
(none)	

Logic

- Rate = rate + delta; then clampLoanInterest.

executeEvent

```
void executeEvent(GameState game[], int playerIndex)
```

Draws the current National Event Card (0..19, cycling like draw-top-put-bottom). Effects are global, so playerIndex is unused.

Actual source code

```
/* Appendix A: 20 National Event Cards, drawn when a player lands on
   an EVENT square. currentCardIndex cycles 0-19 and wraps around,
   which behaves the same as "draw the top card, put it at the
   bottom" without needing a real deck data structure. */
void executeEvent(GameState game[], int playerIndex)
{
    (void)playerIndex;    /* not needed - these effects are global, not per-player */

    printf("\n*** NATIONAL EVENT CARD ***\n");

    switch(game[0].economy.currentCardIndex)
    {
        case 0:
            printf("Tourism Hype : Hotels earn double rent for 5 rounds.\n");
            addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 5);
            break;

        case 1:
            printf("Fuel Shortage : Railway rent doubles for 5 rounds.\n");
            addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 5);
            break;

        case 2:
            printf("Heavy Floods : a random coastal property is damaged.\n");
            triggerDisaster(game);
            break;

        case 3:
        {
            int candidates[BOARD_SIZE];
            int count;
            int i;

            count = 0;

            for(i = 0; i < BOARD_SIZE; i++)
            {
                if(game[0].board[i].type == PROPERTY && game[0].board[i].property.owner != -1)
                {
                    candidates[count] = i;
                    count++;
                }
            }
        }
    }
}
```

```

    }

    if(count > 0)
    {
        game[0].economy.closedPropertyIndex = candidates[rand() % count];
        game[0].economy.closedPropertyRoundsLeft = 2;

        printf("Political Rally : %s is closed for 2 rounds.\n",
            game[0].board[game[0].economy.closedPropertyIndex].name);
    }
    else
    {
        printf("Political Rally : no owned property to close.\n");
    }
    break;
}

case 4:
    printf("Stock Market Rise : All property values increase by 10%% for 15 rounds.\n");
    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);
    break;

case 5:
    printf("Economic Downturn : Property values decrease by 15%% for 15 rounds.\n");
    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
    break;

case 6:
    printf("Housing Subsidy : House construction cost reduced by 30%% for 15 rounds.\n");
    addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 15);
    break;

case 7:
    adjustLoanInterest(game, -LOAN_INTEREST_STEP);

    printf("Interest Rate Cut : Loan interest reduced by %d%%. Now %d%%.\n",
        LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
    break;

case 8:
    adjustLoanInterest(game, LOAN_INTEREST_STEP);

    printf("Interest Rate Increase : Loan interest increased by %d%%. Now %d%%.\n",
        LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
    break;

case 9:
{
    int i;

    printf("Tax Amnesty : Each player receives LKR 2,000.\n");

    for(i = 0; i < MAX_PLAYERS; i++)
        receiveMoney(game, i, 2000);

    break;
}

case 10:
    printf("Power Failure : Utility income halved for 3 rounds.\n");
    addModifier(game, MOD_UTIL_RENT, -1, -1, 50, 3);
    break;

```

```

case 11:
    printf("Foreign Funding : Commercial property values increase by 15%% for 15 rounds.\n");
    addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 115, 15);
    break;

case 12:
    printf("Port Expansion : Railway station values increase by 20%% for 15 rounds.\n");
    addModifier(game, MOD_RAIL_VALUE, -1, -1, 120, 15);
    break;

case 13:
    printf("Festival Season : Hotels receive 50%% additional rent for 5 rounds.\n");
    addModifier(game, MOD_HOTEL_RENT, -1, -1, 150, 5);
    break;

case 14:
    printf("Labour Strike : Construction suspended for 2 rounds.\n");
    game[0].economy.constructionSuspendedRoundsLeft = 2;
    break;

case 15:
    printf("Insurance Discount : Premiums reduced by 20%% for 10 rounds.\n");
    addModifier(game, MOD_INSURANCE, -1, -1, 80, 10);
    break;

case 16:
{
    PropertyGroup group;
    char groupBuffer[32];

    group = (PropertyGroup)(rand() % NO_GROUP);
    groupName(group, groupBuffer);

    printf("Property Revaluation : %s properties appreciate by 15%% for 15 rounds.\n",
        groupBuffer);
    addModifier(game, MOD_GROUP_VALUE, group, -1, 115, 15);
    break;
}

case 17:
    printf("Currency Depreciation : Construction costs increase by 10%% for 15 rounds.\n");
    addModifier(game, MOD_CONSTRUCTION, -1, -1, 110, 15);
    break;

case 18:
{
    int lucky;

    /* Section 1.3: bankrupt players take no further part in the
       game, so a grant must go to a solvent player. Pick a
       random starting point and scan forward to the first
       solvent player - at least one must exist, because a live
       player drew this card. */
    lucky = rand() % MAX_PLAYERS;

    while(game[0].players[lucky].bankrupt)
        lucky = (lucky + 1) % MAX_PLAYERS;

    printf("Government Grant : %s receives LKR 5,000.\n",
        game[0].players[lucky].name);
}

```

```

        receiveMoney(game, lucky, 5000);
        break;
    }

    case 19:
        printf("National Disaster : a random developed property is damaged.\n");
        triggerDisaster(game);
        break;

    default:
        break;
}

game[0].economy.currentCardIndex = (game[0].economy.currentCardIndex + 1) % EVENT_CARD_COUNT;
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	Unused - effects are global, not per-player.

Local variables

Name	Meaning
candidates, count, i	Owned properties for Political Rally case 3.
group, groupBuffer	Random group for Property Revaluation case 16.
lucky	Solvent grant recipient for case 18.

Logic

- Tourism Hype (0) hotels x2 rent 5r; Fuel Shortage (1) rail rent x2 5r; Heavy Floods (2) disaster; Political Rally (3) closes a random owned property 2r; Stock Market Rise (4) values +10% 15r; Economic Downturn (5) values -15% 15r; Housing Subsidy (6) construction -30% 15r; Interest Cut/Rise (7/8) +/-2 points; Tax Amnesty (9) +2000 to everyone; Power Failure (10) utility rent /2 3r; Foreign Funding (11) Orange values +15% 15r; Port Expansion (12) rail values +20% 15r; Festival Season (13) hotel rent +50% 5r; Labour Strike (14) construction suspended 2r; Insurance Discount (15) premiums -20% 10r; Property Revaluation (16) random group +15% 15r; Currency Depreciation (17) construction +10% 15r; Government Grant (18) 5000 to a SOLVENT player (bug fix - never a bankrupt one); National Disaster (19) disaster.
- Advances currentCardIndex = (index + 1) % 20 after drawing.

triggerEconomicEvent

```
void triggerEconomicEvent(GameState game[])
```

One of 8 random economy-wide events every 15 rounds (Tourism Boom, Fuel Crisis, Heavy Monsoon, Economic Recession, Stock Market Boom, Government Housing, Foreign Investment, Political Unrest).

Actual source code

```

/* Section 2.5: one of 8 events, every 15 rounds, affecting every player. */
void triggerEconomicEvent(GameState game[])
{
    int choice;

    choice = rand() % ECONOMIC_EVENT_COUNT;

    printf("\n=== Economic Event ===\n");

    switch(choice)
    {

```

```

case 0:
    printf("Tourism Boom\n");
    printf("Hotels receive double rent for 15 rounds.\n");
    printf("Southern Province properties increase in value by 15%%.\n");

    addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 15);
    addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 115, 15);
    break;

case 1:
    printf("Fuel Crisis\n");
    printf("Railway rent doubles for 15 rounds.\n");
    printf("Property development costs increase 20%% for 15 rounds.\n");

    addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 15);
    addModifier(game, MOD_CONSTRUCTION, -1, -1, 120, 15);
    break;

case 2:
    printf("Heavy Monsoon\n");
    printf("Insurance premiums increase for 15 rounds.\n");
    printf("Coastal properties lose 10%% value.\n");

    addModifier(game, MOD_INSURANCE, -1, -1, 115, 15);
    addModifier(game, MOD_FLOOD_RISK, -1, -1, 100, 15);
    addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 90, 15);
    break;

case 3:
    printf("Economic Recession\n");
    printf("Property values decrease 15%. Rent decreases 10%%.\n");
    printf("Loan interest increases by 15%%.\n");

    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
    addModifier(game, MOD_RENT_GLOBAL, -1, -1, 90, 15);
    addModifier(game, MOD RECESSION, -1, -1, 100, 15);

    game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, 15);
    clampLoanInterest(game);
    break;

case 4:
    printf("Stock Market Boom\n");
    printf("Property values increase 10%. Loan interest decreases 10%%.\n");

    addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);

    game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, -10);
    clampLoanInterest(game);
    break;

case 5:
    printf("Government Housing Programme\n");
    printf("House construction costs reduce 25%%.\n");

    addModifier(game, MOD_CONSTRUCTION, -1, -1, 75, 15);
    break;

case 6:
    printf("Foreign Investment\n");
    printf("Commercial properties increase 20%%.\n");

```

```

        addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 120, 15);
        break;

    case 7:
        printf("Political Unrest\n");
        printf("Hotel rent drops by 50%% for 15 rounds.\n");

        addModifier(game, MOD_HOTEL_RENT, -1, -1, 50, 15);
        addModifier(game, MOD_RIOT_RISK, -1, -1, 100, 15);
        addModifier(game, MOD_BI_CLAIMS, -1, -1, 100, 15);
        break;

    default:
        break;
}
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
choice	rand() % 8.

Logic

- Each event adds timed modifiers; the recession also applies a +15% loan-interest move, and several events boost disaster risks.

triggerGovernmentRegulation

```
void triggerGovernmentRegulation(GameState game[])
```

One of 8 random government regulations every 20 rounds (Property Tax raise, Loan Interest cut, Housing Subsidy, Luxury Property Tax on hotels, Railway Modernization, Electricity Tariff, Insurance Regulation, Anti-Speculation Act).

Actual source code

```

/* Section 2.7: one of 8 regulations, every 20 rounds. */
void triggerGovernmentRegulation(GameState game[])
{
    int choice;

    choice = rand() % REGULATION_COUNT;

    printf("\n=== Government Regulation ===\n");

    switch(choice)
    {
        case 0:
            game[0].economy.incomeTaxRate = applyRate(game[0].economy.incomeTaxRate, 50);

            if(game[0].economy.incomeTaxRate > TAX_RATE_MAX)
                game[0].economy.incomeTaxRate = TAX_RATE_MAX;

            printf("Increase Property Tax\n");
            printf("Income Tax rate increased by 50%%. Now %d%%.\n", game[0].economy.incomeTaxRate);
            break;

        case 1:
            adjustLoanInterest(game, -LOAN_INTEREST_STEP);
    }
}

```

```

        printf("Reduce Loan Interest\n");
        printf("Interest decreased by %d%%. Now %d%%.\n",
            LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
        break;

case 2:
    printf("Housing Subsidy\n");
    printf("House construction costs reduced by 30%% for 20 rounds.\n");

    addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 20);
    break;

case 3:
{
    int i;
    int tax;

    printf("Luxury Property Tax\n");
    printf("Hotels pay a maintenance tax of 25%% of their value.\n");

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY && game[0].board[i].property.hotel &&
            game[0].board[i].property.owner != -1)
        {
            tax = (currentMarketValue(game, i) * HOTEL_LUXURY_TAX_PERCENT) / 100;

            printf("%s pays LKR %d luxury tax on %s.\n",
                game[0].players[game[0].board[i].property.owner].name,
                tax, game[0].board[i].name);

            payMoney(game, game[0].board[i].property.owner, tax);
        }
    }
    break;
}

case 4:
    printf("Railway Modernization\n");
    printf("Railway rents increase 25%% for 20 rounds.\n");

    addModifier(game, MOD_RAIL_RENT, -1, -1, 125, 20);
    break;

case 5:
    printf("Electricity Tariff Revision\n");
    printf("Utility rents increase 20%% for 20 rounds.\n");

    addModifier(game, MOD_UTIL_RENT, -1, -1, 120, 20);
    break;

case 6:
    printf("Insurance Regulation\n");
    printf("Insurance premiums decrease 15%% for 20 rounds. Coverage unchanged.\n");

    addModifier(game, MOD_INSURANCE, -1, -1, 85, 20);
    break;

case 7:
    printf("Anti-Speculation Act\n");
    printf("Players may now own at most three undeveloped properties.\n");

```

```

        game[0].economy.antiSpeculationActive = 1;
        break;

    default:
        break;
}
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
choice	rand() % 8.
tax	Luxury tax for one hotel.
i	Board index.

Logic

- Case 3 (Luxury Tax) bills 25% of each hotel's market value, printed per owner/property (bug fix).
- Case 7 activates the Anti-Speculation Act economy-wide.

decrementEventTimers

```
void decrementEventTimers(GameState game[])
```

Ticks the non-modifier event timers down: construction suspension and the closed property.

Actual source code

```

/* Decrements the non-modifier event timers - the modifier list itself
   is ticked down separately by decrementModifiers(). */
void decrementEventTimers(GameState game[])
{
    if(game[0].economy.constructionSuspendedRoundsLeft > 0)
        game[0].economy.constructionSuspendedRoundsLeft--;

    if(game[0].economy.closedPropertyRoundsLeft > 0)
    {
        game[0].economy.closedPropertyRoundsLeft--;
        if(game[0].economy.closedPropertyRoundsLeft == 0)
            game[0].economy.closedPropertyIndex = -1;
    }
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
(none)	Mutates economy fields.

Logic

- Modifier timers are decremented separately (decrementModifiers).

4.8 economy.c

Inflation, the modifier economy, ageing and maintenance. `currentMarketValue` is the single 'real price' function reused across the whole game.

File header (includes / comments)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"
```

initEconomy

```
void initEconomy(GameState game[])
```

Resets all economy state: zero inflation, base rates, empty modifier list.

Actual source code

```
void initEconomy(GameState game[])
{
    game[0].economy.inflationRate = 0;
    game[0].economy.loanInterestRate = LOAN_INTEREST_RATE;

    game[0].economy.constructionSuspendedRoundsLeft = 0;

    game[0].economy.closedPropertyIndex = -1;
    game[0].economy.closedPropertyRoundsLeft = 0;

    game[0].economy.incomeTaxRate = INCOME_TAX_RATE;

    game[0].economy.antiSpeculationActive = 0;

    game[0].economy.currentCardIndex = 0;

    game[0].economy.modifierCount = 0;
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
(none)	

Logic

- Loan interest = `LOAN_INTEREST_RATE`, tax = `INCOME_TAX_RATE`, card index 0, no modifiers, anti-spec off, no closed property.

applyRate

```
int applyRate(int oldValue, int ratePercent)
```

Rule-LK 14: applies a percentage change, never returning below 1.

Actual source code

```

/* Rule-LK 14: New = Old x (1 + rate/100), never below 1. */
int applyRate(int oldValue, int ratePercent)
{
    int newValue;

    newValue = oldValue + (oldValue * ratePercent) / 100;

    if(newValue < 1)
        newValue = 1;

    return newValue;
}

```

Return value

Name	Meaning
returns	New value (≥ 1).

Parameters

Name	Meaning
oldValue	Starting value.
ratePercent	Signed percent (-3, +8, ...).

Local variables

Name	Meaning
newValue	Computed value.

Logic

- $newValue = oldValue + oldValue \times rate/100$; clamped at 1. Used by inflation and interest moves.

addSourcedModifier

```

void addSourcedModifier(GameState game[], ModifierType type, int group, int index, int percent, int
roundsLeft, ModifierSource source)

```

Appends a timed modifier to the economy, tagging its source for Rule-LK 36 reporting.

Actual source code

```

/* Adds a timed modifier (Rule-LK 30-35). group/index are -1 when not
relevant. Effects accumulate: two active +25% rents give
roughly +56%, so simultaneous effects are cumulative (Rule-LK 34). */
void addSourcedModifier(GameState game[], ModifierType type, int group,
                        int index, int percent, int roundsLeft,
                        ModifierSource source)
{
    if(game[0].economy.modifierCount >= MAX_MODIFIERS)
        return;

    game[0].economy.modifiers[game[0].economy.modifierCount].type = type;
    game[0].economy.modifiers[game[0].economy.modifierCount].group = group;
    game[0].economy.modifiers[game[0].economy.modifierCount].index = index;
    game[0].economy.modifiers[game[0].economy.modifierCount].percent = percent;
    game[0].economy.modifiers[game[0].economy.modifierCount].roundsLeft = roundsLeft;
    game[0].economy.modifiers[game[0].economy.modifierCount].source = source;

    game[0].economy.modifierCount++;
}

```

Return value

Name	Meaning
------	---------

returns void.

Parameters

Name	Meaning
type	What it affects.
group	Group scope or -1.
index	Index scope or -1.
percent	Multiplier (100 = no change).
roundsLeft	Duration.
source	SRC_GENERAL/BOOM/DECLINE/REGIONAL.

Local variables

Name	Meaning
(none)	Writes into modifiers[modifierCount].

Logic

- Ignores the add when modifierCount == MAX_MODIFIERS (48).

addModifier

```
void addModifier(GameState game[], ModifierType type, int group, int index, int percent, int roundsLeft)
```

addSourcedModifier with SRC_GENERAL (default for events/cards/regulations).

Actual source code

```
/* Events, national cards and regulations use this - they are all
   generic effects, not Market Boom/Decline or Regional cards. */
void addModifier(GameState game[], ModifierType type, int group, int index,
                 int percent, int roundsLeft)
{
    addSourcedModifier(game, type, group, index, percent, roundsLeft, SRC_GENERAL);
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
type	What it affects.
group	Group scope or -1.
index	Index scope or -1.
percent	Multiplier.
roundsLeft	Duration.

Local variables

Name	Meaning
(none)	Delegates.

Logic

- Convenience wrapper - market and regional calls use the sourced version.

modifierMultiplier

```
int modifierMultiplier(GameState game[], ModifierType type, int group, int index)
```

Multiplies every matching active modifier together (Rule-LK 34) - so two +25% are roughly +56%.

Actual source code

```
/* Effective percentage multiplier from every active matching modifier,
multiplied together (Rule-LK 34). Returns 100 when none match.
A modifier with group == -1 is global and applies to every group,
so it matches regardless of the requested group (and likewise for
index == -1). */
int modifierMultiplier(GameState game[], ModifierType type, int group, int index)
{
    int mult;
    int i;

    mult = 100;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        if(game[0].economy.modifiers[i].type != type)
            continue;
        if(group != -1 && game[0].economy.modifiers[i].group != -1 &&
            game[0].economy.modifiers[i].group != group)
            continue;
        if(index != -1 && game[0].economy.modifiers[i].index != -1 &&
            game[0].economy.modifiers[i].index != index)
            continue;

        mult = (mult * game[0].economy.modifiers[i].percent) / 100;
    }

    return mult;
}
```

Return value

Name	Meaning
returns	Combined multiplier, 100 = no change.

Parameters

Name	Meaning
type	Modifier kind.
group	Requested group (-1 = any).
index	Requested index (-1 = any).

Local variables

Name	Meaning
mult	Running product.
i	Modifier index.

Logic

- A stored group == -1 is global and matches every requested group; likewise for index. Matching modifiers chain-multiply.

isModifierActive

```
int isModifierActive(GameState game[], ModifierType type, int group, int index)
```

Whether at least one matching flag-style modifier (flood risk, recession, BI claims) is active.

Actual source code

```

/* Whether at least one matching modifier is currently active - used
for flag-style modifiers such as flood risk or recession. */
int isModifierActive(GameState game[], ModifierType type, int group, int index)
{
    int i;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        if(game[0].economy.modifiers[i].type != type)
            continue;
        if(group != -1 && game[0].economy.modifiers[i].group != group)
            continue;
        if(index != -1 && game[0].economy.modifiers[i].index != index)
            continue;

        return 1;
    }

    return 0;
}

```

Return value

Name	Meaning
returns	1 = active, 0 = not.

Parameters

Name	Meaning
type	Modifier kind.
group	Group scope or -1.
index	Index scope or -1.

Local variables

Name	Meaning
i	Modifier index.

Logic

- Returns at the first exact match - used to gate weighted disasters and strategy decisions.

currentTaxRatePercent

```
int currentTaxRatePercent(GameState game[], int baseRate)
```

The effective tax rate: base rate scaled by MOD_VALUE_GLOBAL.

Actual source code

```

/* The effective tax rate used by the Income Tax and Community
Development Fund squares. Starts at the given base rate (15% /
10%) and is scaled by the currently active global value
modifiers, so market fluctuations move the rate up or down. */
int currentTaxRatePercent(GameState game[], int baseRate)
{
    return (baseRate * modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1)) / 100;
}

```

Return value

Name	Meaning
returns	Effective percent.

Parameters

Name	Meaning
baseRate	Base rate (15 or 10).

Local variables

Name	Meaning
(none)	Delegates to modifierMultiplier.

Logic

- Used by both Income Tax and Community Fund purposes.

decrementModifiers

```
void decrementModifiers(GameState game[])
```

Ticks every active modifier down and compacts out expired ones.

Actual source code

```
/* Ticks every active modifier down and removes the expired ones. */
void decrementModifiers(GameState game[])
{
    int i;

    i = 0;

    while(i < game[0].economy.modifierCount)
    {
        game[0].economy.modifiers[i].roundsLeft--;

        if(game[0].economy.modifiers[i].roundsLeft <= 0)
        {
            int j;

            for(j = i; j < game[0].economy.modifierCount - 1; j++)
                game[0].economy.modifiers[j] = game[0].economy.modifiers[j + 1];

            game[0].economy.modifierCount--;
        }
        else
        {
            i++;
        }
    }
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Scan index.
j	Compaction shift.

Logic

- In-place: when roundsLeft <= 0 the array is shifted left, otherwise i advances.

formatLKR

```
void formatLKR(int amount, char out[])
```

Formats an amount with thousands separators, e.g. 30000 -> "30,000".

Actual source code

```
/* Formats an amount with thousands separators, e.g. 30000 -> "30,000". */
void formatLKR(int amount, char out[])
{
    char digits[32];
    int len;
    int i;
    int w;
    int neg;

    neg = 0;

    if(amount < 0)
    {
        neg = 1;
        amount = -amount;
    }

    sprintf(digits, "%d", amount);
    len = strlen(digits);

    w = 0;

    if(neg)
        out[w++] = '-';

    for(i = 0; i < len; i++)
    {
        if(i > 0 && (len - i) % 3 == 0)
            out[w++] = ',';

        out[w++] = digits[i];
    }

    out[w] = '\0';
}
```

Return value

Name	Meaning
returns	void (writes into out).

Parameters

Name	Meaning
amount	Raw integer.
out	Destination buffer.

Local variables

Name	Meaning
digits	Plain digits.
len	Digit count.
i	Digit index.
w	Writer index.
neg	Negative flag.

Logic

- Inserts a comma every 3 digits from the right and preserves a leading '-'.

applyInflation

```
void applyInflation(GameState game[])
```

Every 10 rounds picks a random rate {-3, 0, 2, 5, 8, 12} and applies it to purchase prices, mortgage values, rents, construction costs and the loan interest rate.

Actual source code

```
/* Every 10 rounds, a random inflation rate is applied to every
   property's price, rent, and construction costs. */
void applyInflation(GameState game[])
{
    int possibleRates[6] = {-3, 0, 2, 5, 8, 12};
    int rate;
    int i;

    rate = possibleRates[rand() % 6];
    game[0].economy.inflationRate = rate;

    printf("\n=== Inflation Update ===\n");
    printf("New Inflation Rate : %d%%\n", rate);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {
            continue;
        }

        game[0].board[i].property.purchasePrice =
            applyRate(game[0].board[i].property.purchasePrice, rate);

        game[0].board[i].property.mortgageValue =
            applyRate(game[0].board[i].property.mortgageValue, rate);

        if(game[0].board[i].type == PROPERTY)
        {
            game[0].board[i].property.baseRent =
                applyRate(game[0].board[i].property.baseRent, rate);

            game[0].board[i].property.houseCost =
                applyRate(game[0].board[i].property.houseCost, rate);

            game[0].board[i].property.hotelCost =
                applyRate(game[0].board[i].property.hotelCost, rate);
        }
    }

    game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, rate);

    printf("New Loan Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
}
```

Return value

Name	Meaning
------	---------

returns void.

Local variables

Name	Meaning
possibleRates	The draw set.
rate	Chosen rate.
i	Board index.

Logic

- Uses applyRate on every ownable square's price/mortgage, plus property rent, house cost and hotel cost; then the loan interest rate.

currentMarketValue

```
int currentMarketValue(GameState game[], int propIndex)
```

The real worth of a property: purchase price minus age depreciation, times every applicable value modifier (global, group, rail, index).

Actual source code

```
/* A property's real current worth: purchase price, minus age
depreciation, adjusted by any active market conditions. Reused
everywhere a "real price" is needed - buying, insurance, repairs,
renovation, net worth - so all of them stay consistent. */
/* Rule-LK 11-13, 24: the current market value of a property, after
age depreciation and every value modifier currently active
(Rule-LK 34 - all matching modifiers multiply together). */
int currentMarketValue(GameState game[], int propIndex)
{
    int price;
    int depreciation;
    int mult;

    price = game[0].board[propIndex].property.purchasePrice;
    depreciation = game[0].board[propIndex].property.depreciation;

    price = price - (price * depreciation) / 100;

    mult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

    if(game[0].board[propIndex].type == PROPERTY)
        mult = (mult * modifierMultiplier(game, MOD_GROUP_VALUE,
                                           game[0].board[propIndex].property.group, -1)) / 100;

    if(game[0].board[propIndex].type == RAILWAY)
        mult = (mult * modifierMultiplier(game, MOD_RAIL_VALUE, -1, -1)) / 100;

    mult = (mult * modifierMultiplier(game, MOD_INDEX_VALUE, -1, propIndex)) / 100;

    price = (price * mult) / 100;

    return price;
}
```

Return value

Name	Meaning
returns	Value in LKR.

Parameters

Name	Meaning
------	---------

propIndex The square.

Local variables

Name	Meaning
price	Working price.
depreciation	Age depreciation percent.
mult	Combined modifier multiplier.

Logic

- Base = purchasePrice x (100 - depreciation)/100.
- Mult accumulates MOD_VALUE_GLOBAL, then MOD_GROUP_VALUE (property), MOD_RAIL_VALUE (railway), then MOD_INDEX_VALUE (this square).
- Reused by buying, insurance, repairs, auctions, net worth - the single source of truth for prices.

ageProperties

```
void ageProperties(GameState game[])
```

Rule-LK 15/16: ages every property; past age 50 depreciation ticks +5% every 5 rounds, capped at 30%, printing when it changes.

Actual source code

```
/* Rule-LK 15, 16: properties get older every round, and lose value
   past age 50. Prints the required "Property Depreciation" message
   (Section 5) whenever a property's depreciation actually changes. */
void ageProperties(GameState game[])
{
    int i;
    int roundsOver50;
    int newDepreciation;
    int oldDepreciation;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        game[0].board[i].property.age++;

        if(game[0].board[i].property.age > PROPERTY_AGE_LIMIT)
        {
            oldDepreciation = game[0].board[i].property.depreciation;

            roundsOver50 = game[0].board[i].property.age - PROPERTY_AGE_LIMIT;
            newDepreciation = roundsOver50 / DEPRECIATION_STEP;

            if(newDepreciation > MAX_DEPRECIATION)
                newDepreciation = MAX_DEPRECIATION;

            game[0].board[i].property.depreciation = newDepreciation;

            if(newDepreciation != oldDepreciation)
            {
                printf("\nProperty Depreciation\n");
                printf("Property\n%s\n", game[0].board[i].name);
                printf("has depreciated by %d%%.\n", newDepreciation);
                printf("Current Value\nLKR %d\n", currentMarketValue(game, i));
            }
        }
    }
}
```

```
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Board index.
roundsOver50	Age above the limit.
newDepreciation	Computed depreciation.
oldDepreciation	Previous value.

Logic

- Depreciation = roundsOver50 / 5, capped at MAX_DEPRECIATION (30).

tryRenovateAgeDepreciation

```
void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
```

Rule-LK 17: landing on your own aged property offers a renovation - 10% of value, resetting age/depreciation and raising base rent +5%.

Actual source code

```
/* Rule-LK 17: landing on your own aged property offers a renovation. */
void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
{
    int cost;

    if(game[0].board[propIndex].property.depreciation <= 0)
        return;

    if(!shouldRenovateAgeDepreciation(game, playerIndex,
                                      game[0].board[propIndex].property.depreciation))
    {
        return;
    }

    cost = (currentMarketValue(game, propIndex) * RENOVATION_COST_PERCENT) / 100;

    if(game[0].players[playerIndex].cash < cost)
        return;

    payMoney(game, playerIndex, cost);

    game[0].board[propIndex].property.age = 0;
    game[0].board[propIndex].property.depreciation = 0;

    game[0].board[propIndex].property.baseRent =
        applyRate(game[0].board[propIndex].property.baseRent, RENOVATION_RENT_BOOST);

    printf("\n%s renovated %s.\n",
        game[0].players[playerIndex].name, game[0].board[propIndex].name);

    printf("Renovation Cost : LKR %d\n", cost);
}
```

Return value

Name	Meaning
------	---------

returns void.

Parameters

Name	Meaning
playerIndex	The owner.
propIndex	The property.

Local variables

Name	Meaning
cost	Renovation cost.

Logic

- Only if there is depreciation, the strategy agrees, and the player can afford it.

rentConditionPercent

```
int rentConditionPercent(int condition)
```

Table 3: maps building condition to the fraction of rent collected.

Actual source code

```
/* Table 3: rent collected as a percentage of building condition. */
int rentConditionPercent(int condition)
{
    if(condition >= 90)
        return 100;
    if(condition >= 75)
        return 90;
    if(condition >= 50)
        return 75;
    if(condition >= 25)
        return 50;

    return 0; /* below 25% - building closed */
}
```

Return value

Name	Meaning
returns	Percent (0, 50, 75, 90 or 100).

Parameters

Name	Meaning
condition	0..100.

Local variables

Name	Meaning
(none)	Branches on thresholds.

Logic

- >=90 -> 100%, >=75 -> 90, >=50 -> 75, >=25 -> 50, else 0 (closed).

ageBuildings

```
void ageBuildings(GameState game[])
```

Rule-LK 25/28: condition drops 2 per round; lost-income countdown runs; 20+ rounds without maintenance causes structural damage (-15% value, -25% rent, 150% repairs).

Actual source code

```

/* Rule-LK 25, 28: building condition wears down every round, and
   20+ rounds of neglect causes lasting structural damage. */
void ageBuildings(GameState game[])
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        /* Business Interruption's 5-round lost-income countdown runs
           even if the property isn't currently developed with houses
           (it still has a hotel that just isn't earning anything). */
        if(game[0].board[i].property.lostIncomeRoundsLeft > 0)
            game[0].board[i].property.lostIncomeRoundsLeft--;

        if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
            continue;

        game[0].board[i].property.condition -= 2;

        if(game[0].board[i].property.condition < 0)
            game[0].board[i].property.condition = 0;

        game[0].board[i].property.roundsSinceMaintenance++;

        if(game[0].board[i].property.roundsSinceMaintenance > NEGLECT_ROUNDS &&
            !game[0].board[i].property.structurallyDamaged)
        {
            game[0].board[i].property.structurallyDamaged = 1;

            game[0].board[i].property.preDamagePurchasePrice =
                game[0].board[i].property.purchasePrice;
            game[0].board[i].property.preDamageBaseRent =
                game[0].board[i].property.baseRent;

            game[0].board[i].property.purchasePrice =
                game[0].board[i].property.purchasePrice -
                (game[0].board[i].property.purchasePrice * STRUCTURAL_VALUE_LOSS) / 100;

            game[0].board[i].property.baseRent =
                game[0].board[i].property.baseRent -
                (game[0].board[i].property.baseRent * STRUCTURAL_RENT_LOSS) / 100;

            game[0].board[i].property.maintenanceCostMultiplierPercent = DAMAGE_REPAIR_MULTIPLIER;
            game[0].board[i].property.condition = 0;

            printf("\n%s has suffered structural damage from neglect!\n",
                game[0].board[i].name);
        }
    }
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
------	---------

Logic

- Business Interruption countdown decrements even for undeveloped properties (a hotel exists).
- On structural damage: stores pre-damage price/rent, applies losses, sets the cost multiplier to 150% and condition to 0.

performMaintenance

```
void performMaintenance(GameState game[], int playerIndex)
```

Rule-LK 27: restores a building's condition to 100% for a fee (hotel 8%, house 5%), if the strategy agrees.

Actual source code

```
/* Rule-LK 27: restores a building's condition to 100% for a fee. */
void performMaintenance(GameState game[], int playerIndex)
{
    int i;
    int baseCost;
    int cost;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
            continue;

        if(game[0].board[i].property.structurallyDamaged)
            continue; /* needs a full renovation instead */

        if(game[0].board[i].property.condition >= 100)
            continue;

        if(game[0].board[i].property.hotel)
            baseCost = (game[0].board[i].property.hotelCost * 8) / 100;
        else
            baseCost = (game[0].board[i].property.houseCost * 5) / 100;

        cost = (baseCost * game[0].board[i].property.maintenanceCostMultiplierPercent) / 100;

        if(!shouldMaintain(game, playerIndex, game[0].board[i].property.condition, cost))
            continue;

        if(game[0].players[playerIndex].cash < cost)
            continue;

        payMoney(game, playerIndex, cost);

        game[0].board[i].property.condition = 100;
        game[0].board[i].property.roundsSinceMaintenance = 0;

        printf("\n%s performed maintenance on %s.\n",
            game[0].players[playerIndex].name, game[0].board[i].name);

        printf("Maintenance Cost : LKR %d\n", cost);
    }
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The owner.

Local variables

Name	Meaning
i	Board index.
baseCost	Percent of building cost.
cost	Cost x maintenance multiplier.

Logic

- Skips undeveloped, structurally damaged or condition-100 buildings; resets roundsSinceMaintenance.

renovateStructuralDamage

```
void renovateStructuralDamage(GameState game[], int playerIndex)
```

Rule-LK 29: fully restores a structurally damaged building at 150% of replacement value, restoring pre-damage price/rent.

Actual source code

```
/* Rule-LK 29: fully restores a structurally damaged building. */
void renovateStructuralDamage(GameState game[], int playerIndex)
{
    int i;
    int replacementValue;
    int cost;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(!game[0].board[i].property.structurallyDamaged)
            continue;

        if(game[0].board[i].property.hotel)
            replacementValue = game[0].board[i].property.hotelCost;
        else
            replacementValue = game[0].board[i].property.houseCost;

        cost = (replacementValue * DAMAGE_REPAIR_MULTIPLIER) / 100;

        if(game[0].players[playerIndex].cash < cost)
            continue;

        payMoney(game, playerIndex, cost);

        game[0].board[i].property.purchasePrice = game[0].board[i].property.preDamagePurchasePrice;
        game[0].board[i].property.baseRent = game[0].board[i].property.preDamageBaseRent;
        game[0].board[i].property.condition = 100;
        game[0].board[i].property.structurallyDamaged = 0;
        game[0].board[i].property.maintenanceCostMultiplierPercent = 100;
    }
}
```

```

        game[0].board[i].property.roundsSinceMaintenance = 0;

        printf("\n%s fully renovated %s after structural damage.\n",
            game[0].players[playerIndex].name, game[0].board[i].name);

        printf("Renovation Cost : LKR %d\n", cost);
    }
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The owner.

Local variables

Name	Meaning
i	Board index.
replacementValue	Hotel or house cost.
cost	150% of replacement value.

Logic

- Restores preDamagePurchasePrice/preDamageBaseRent, condition 100, multiplier 100, clears the damage flag.

4.9 insurance.c

Policies, disasters and repairs. Insurance squares sell policies whose type decides coverage and compensation; triggerDisaster runs the damage flow every 10 rounds.

File header (includes / comments)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

```

propertyValue

```
int propertyValue(GameState game[], int propIndex)
```

A property's value = currentMarketValue.

Actual source code

```

int propertyValue(GameState game[], int propIndex)
{
    return currentMarketValue(game, propIndex);
}

```

Return value

Name	Meaning
returns	Value in LKR.

Parameters

Name	Meaning
------	---------

propIndex	The square.
-----------	-------------

Local variables

Name	Meaning
(none)	Delegates.

Logic

- Simple alias so insurance code reads naturally.

repairCost

```
int repairCost(GameState game[], int propIndex)
```

Assumption: repairs cost 30% of a property's value.

Actual source code

```
/* Assumption: repair costs 30% of the property's value. The
assignment doesn't give an exact figure, so this is used
consistently everywhere repair cost is needed. */
int repairCost(GameState game[], int propIndex)
{
    return (propertyValue(game, propIndex) * REPAIR_COST_PERCENT) / 100;
}
```

Return value

Name	Meaning
returns	Cost in LKR.

Parameters

Name	Meaning
propIndex	The square.

Local variables

Name	Meaning
(none)	Uses REPAIR_COST_PERCENT.

Logic

- $\text{propertyValue} \times 30 / 100$, used consistently everywhere repair cost is needed.

isCovered

```
int isCovered(InsuranceType policy, DisasterType disaster)
```

Whether a policy covers a disaster type.

Actual source code

```
int isCovered(InsuranceType policy, DisasterType disaster)
{
    if(policy == NO_INSURANCE)
        return 0;

    if(policy == BASIC_INSURANCE)
        return (disaster == FIRE || disaster == FLOOD);

    return 1; /* Comprehensive and Business Interruption cover everything */
}
```

Return value

Name	Meaning
------	---------

returns 1 = covered, 0 = not.

Parameters

Name	Meaning
policy	Held policy.
disaster	The type that happened.

Local variables

Name	Meaning
(none)	Branches on policy.

Logic

- Basic covers only Fire + Flood; Comprehensive and Business Interruption cover everything.

compensationPercent

```
int compensationPercent(InsuranceType policy)
```

What fraction of the repair cost a policy pays.

Actual source code

```
int compensationPercent(InsuranceType policy)
{
    if(policy == BASIC_INSURANCE)
        return 80;

    if(policy == COMPREHENSIVE_INSURANCE || policy == BUSINESS_INTERRUPTION)
        return 100;

    return 0;
}
```

Return value

Name	Meaning
returns	Percent (0, 80 or 100).

Parameters

Name	Meaning
policy	Held policy.

Local variables

Name	Meaning
(none)	

Logic

- Basic 80%; Comprehensive and Business Interruption 100%.

pickDisaster

```
DisasterType pickDisaster(GameState game[])
```

Weighted random disaster draw; all five start at weight 20, and active flood/riot risks add +30.

Actual source code

```
/* Picks a disaster. Every type is equally likely at base, but Heavy
   Monsoon boosts floods and Political Unrest boosts riots (Rule-LK 34). */
DisasterType pickDisaster(GameState game[])
{
    int weights[5];
    int total;
```

```

int i;
int roll;
DisasterType result;

weights[FIRE] = DISASTER_WEIGHT;
weights[FLOOD] = DISASTER_WEIGHT;
weights[RIOT] = DISASTER_WEIGHT;
weights[BUILDING_COLLAPSE] = DISASTER_WEIGHT;
weights[ELECTRICAL_FAILURE] = DISASTER_WEIGHT;

if(isModifierActive(game, MOD_FLOOD_RISK, -1, -1))
    weights[FLOOD] += DISASTER_BOOST;

if(isModifierActive(game, MOD_RIOT_RISK, -1, -1))
    weights[RIOT] += DISASTER_BOOST;

total = 0;

for(i = 0; i < 5; i++)
    total += weights[i];

roll = rand() % total;

result = FIRE;

for(i = 0; i < 5; i++)
{
    if(roll < weights[i])
    {
        result = (DisasterType)i;
        break;
    }

    roll -= weights[i];
}

return result;
}

```

Return value

Name	Meaning
returns	DisasterType.

Local variables

Name	Meaning
weights	Per-type weight.
total	Sum of weights.
i	Index.
roll	rand() % total.
result	Chosen type.

Logic

- Heavy Monsoon -> flood more likely; Political Unrest -> riot more likely (Rule-LK 34).

purchaseInsurance

```
void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
```

Buys a policy: premium = value x 5/10/15% x insurance modifier; policy lasts 20 rounds.

Actual source code

```
void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
{
    int value;
    int premium;
    int premiumPercent;
    char premiumBuf[32];

    if(type == BASIC_INSURANCE)
        premiumPercent = PREMIUM_BASIC_PERCENT;
    else if(type == COMPREHENSIVE_INSURANCE)
        premiumPercent = PREMIUM_COMPREHENSIVE_PERCENT;
    else if(type == BUSINESS_INTERRUPTION)
        premiumPercent = PREMIUM_INTERRUPTION_PERCENT;
    else
        return;

    value = propertyValue(game, propIndex);
    premium = (value * premiumPercent) / 100;

    /* Rule-LK 34: premiums respond to insurance modifiers. */
    premium = (premium * modifierMultiplier(game, MOD_INSURANCE, -1, -1)) / 100;

    if(game[0].players[playerIndex].cash < premium)
        return;

    payMoney(game, playerIndex, premium);

    game[0].board[propIndex].property.insurance = type;
    game[0].board[propIndex].property.insuranceRoundsLeft = INSURANCE_DURATION_ROUNDS;

    printf("\n%s purchased insurance for %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[propIndex].name);

    formatLKR(premium, premiumBuf);
    printf("Premium : LKR %s.\n", premiumBuf);
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The buyer.
propIndex	The property.
type	Policy to buy.

Local variables

Name	Meaning
value	Property value.
premium	Computed premium.
premiumPercent	Type's rate.
premiumBuf	Formatted premium.

Logic

- Only buys if affordable; sets insurance and insuranceRoundsLeft = 20.

findPropertyToInsure

```
int findPropertyToInsure(GameState game[], int playerIndex)
```

Finds the first owned, uninsured property the strategy actually wants to insure.

Actual source code

```
/* Any owned property can be insured (Section 2.8) - no longer just
   developed ones. */
int findPropertyToInsure(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.insurance != NO_INSURANCE)
            continue;

        if(desiredInsurance(game, playerIndex, i) != NO_INSURANCE)
            return i;
    }

    return -1;
}
```

Return value

Name	Meaning
returns	Square index, or -1.

Local variables

Name	Meaning
i	Board index.

Logic

- Any owned property can be insured (Section 2.8), developed or not - as long as desiredInsurance != NO_INSURANCE.

findPropertyToRenew

```
int findPropertyToRenew(GameState game[], int playerIndex)
```

Finds a policy close to expiring (<= 10 rounds left) for renewal.

Actual source code

```
/* Finds a policy close to expiring so it can be renewed. */
int findPropertyToRenew(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
```

```

        continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.insurance == NO_INSURANCE)
            continue;

        if(game[0].board[i].property.insuranceRoundsLeft <= INSURANCE_RENEW_AT)
            return i;
    }

    return -1;
}

```

Return value

Name	Meaning
returns	Square index, or -1.

Local variables

Name	Meaning
i	Board index.

Logic

- Returns the first insured property under INSURANCE_RENEW_AT.

handleInsuranceVisit

```
void handleInsuranceVisit(GameState game[], int playerIndex)
```

The Insurance square: renew an expiring policy first, otherwise buy a new one per the strategy.

Actual source code

```

void handleInsuranceVisit(GameState game[], int playerIndex)
{
    int propIndex;
    InsuranceType type;

    printf("\n%s landed on an Insurance square.\n",
        game[0].players[playerIndex].name);

    propIndex = findPropertyToRenew(game, playerIndex);

    if(propIndex != -1)
    {
        type = game[0].board[propIndex].property.insurance;

        purchaseInsurance(game, playerIndex, propIndex, type);

        printf("The policy on %s was renewed for another %d rounds.\n",
            game[0].board[propIndex].name, INSURANCE_DURATION_ROUNDS);
        return;
    }

    propIndex = findPropertyToInsure(game, playerIndex);

    if(propIndex == -1)
    {
        printf("No suitable property to insure right now.\n");
        return;
    }
}

```

```

    type = desiredInsurance(game, playerIndex, propIndex);

    purchaseInsurance(game, playerIndex, propIndex, type);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The visitor.

Local variables

Name	Meaning
propIndex	Chosen property.
type	Policy type.

Logic

- Renewal re-buys the same type for a fresh 20 rounds.

tryAutoRepair

```
void tryAutoRepair(GameState game[], int playerIndex)
```

Rule-LK 11: at the start of a turn, repairs any disaster-damaged building the owner can afford.

Actual source code

```

/* Rule-LK 11: fixes a disaster-damaged building automatically once
   the owner can afford it. Called at the start of every turn. */
void tryAutoRepair(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(!game[0].board[i].property.damaged)
            continue;

        if(game[0].players[playerIndex].cash >= game[0].board[i].property.repairCostOwed)
        {
            int cost;
            char costBuf[32];

            cost = game[0].board[i].property.repairCostOwed;

            formatLKR(cost, costBuf);

            payMoney(game, playerIndex, cost);

            printf("\n%s repaired %s. It can collect rent again.\n",
                game[0].players[playerIndex].name, game[0].board[i].name);

            printf("Repair Cost : LKR %s.\n", costBuf);
        }
    }
}

```

```

        game[0].board[i].property.damaged = 0;
        game[0].board[i].property.repairCostOwed = 0;
    }
}
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
playerIndex	The player.

Local variables

Name	Meaning
i	Board index.
cost	repairCostOwed.
costBuf	Formatted cost.

Logic

- Pays repairCostOwed, clears damaged/repairCostOwed so rent resumes.

disasterName

```
void disasterName(DisasterType disaster, char out[])
```

Writes the human name of a disaster into out.

Actual source code

```

/* Writes the name of a disaster into out. */
void disasterName(DisasterType disaster, char out[])
{
    if(disaster == FIRE)
        strcpy(out, "Fire");
    else if(disaster == FLOOD)
        strcpy(out, "Flood");
    else if(disaster == RIOT)
        strcpy(out, "Riot");
    else if(disaster == BUILDING_COLLAPSE)
        strcpy(out, "Building Collapse");
    else
        strcpy(out, "Electrical Failure");
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
disaster	The disaster type.
out	Destination buffer.

Local variables

Name	Meaning
------	---------

(none)

Logic

- Fire/Flood/Riot/Building Collapse/Electrical Failure.

payCompensation

```
int payCompensation(GameState game[], int chosen, int owner, int cost)
```

Pays the insured owner compensation (80%/100%), inflating BI claims x150% during Political Unrest, and sets 5 rounds of lost income for BI policies.

Actual source code

```
/* Pays an insured player's compensation for a disaster and returns
   how much they received. */
int payCompensation(GameState game[], int chosen, int owner, int cost)
{
    int compensation;

    compensation = (cost * compensationPercent(game[0].board[chosen].property.insurance)) / 100;

    /* Political Unrest inflates Business Interruption claims (Rule-LK 34). */
    if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION &&
        isModifierActive(game, MOD_BI_CLAIMS, -1, -1))
    {
        compensation = (compensation * DAMAGE_REPAIR_MULTIPLIER) / 100;
    }

    receiveMoney(game, owner, compensation);

    printf("Insurance Claim Approved.\n");
    printf("Compensation Paid : LKR %d\n", compensation);

    /* Business Interruption also covers 5 rounds of lost rental
       income - the repair cost is fully paid above, but the
       property still earns nothing for 5 rounds. */
    if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION)
    {
        game[0].board[chosen].property.lostIncomeRoundsLeft = BI_LOST_INCOME_ROUNDS;
        printf("%s will earn no rent for the next %d rounds "
            "(Business Interruption).\n", game[0].board[chosen].name,
            BI_LOST_INCOME_ROUNDS);
    }

    return compensation;
}
```

Return value

Name	Meaning
returns	Compensation actually paid.

Parameters

Name	Meaning
chosen	Damaged square.
owner	The owner.
cost	Repair cost.

Local variables

Name	Meaning
------	---------

compensation

Raw compensation.

Logic

- BI policies also set `lostIncomeRoundsLeft = 5` (still no rent for 5 rounds).

triggerDisaster

```
void triggerDisaster(GameState game[])
```

Rule-LK 10: every 10 rounds picks a random developed property and a disaster; insured owners get compensation, uninsured owners are marked `sufferedLoss` and own the full repair cost.

Actual source code

```
/* Rule-LK 10: every 10 rounds, one random developed property may
   be hit by a disaster. */
void triggerDisaster(GameState game[])
{
    int developed[BOARD_SIZE];
    int developedCount;
    int i;
    int chosen;
    DisasterType disaster;
    int owner;
    int cost;
    int compensation;
    char name[32];

    developedCount = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
            (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
        {
            developed[developedCount] = i;
            developedCount++;
        }
    }

    if(developedCount == 0)
        return;

    chosen = developed[rand() % developedCount];

    disaster = pickDisaster(game);

    owner = game[0].board[chosen].property.owner;

    printf("\n*** DISASTER ***\n");

    disasterName(disaster, name);
    printf("%s occurred.\n", name);

    printf("Affected Property : %s (owner : %s)\n",
        game[0].board[chosen].name, game[0].players[owner].name);

    cost = repairCost(game, chosen);

    if(isCovered(game[0].board[chosen].property.insurance, disaster))
    {
        compensation = payCompensation(game, chosen, owner, cost);
    }
}
```

```

    }
    else
    {
        compensation = 0;
        game[0].players[owner].sufferedLoss = 1;

        printf("Property was NOT insured against this disaster.\n");
    }

    game[0].board[chosen].property.repairCostOwed = cost - compensation;

    if(game[0].board[chosen].property.repairCostOwed < 0)
        game[0].board[chosen].property.repairCostOwed = 0;

    if(game[0].board[chosen].property.repairCostOwed > 0)
    {
        game[0].board[chosen].property.damaged = 1;
        printf("%s is damaged and stops earning rent until repaired.\n",
            game[0].board[chosen].name);
    }
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
developed	Indices of developed properties.
developedCount	Their count.
i	Board index.
chosen	Struck property.
disaster	Chosen disaster.
owner	Its owner.
cost	Repair cost.
compensation	Insurance payout.
name	Disaster name.

Logic

- If no developed property exists, nothing happens.
- Uninsured loss sets sufferedLoss = 1 - which flips the Risk Taker's strategy to wanting insurance.
- repairCostOwed = cost - compensation (clamped >= 0); if > 0 the property is damaged (no rent) until auto-repaired.

processInsuranceExpiry

```
void processInsuranceExpiry(GameState game[])
```

Counts every policy down each round; warns at 3 rounds and expires (removes) at 0.

Actual source code

```

void processInsuranceExpiry(GameState game[])
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.insurance == NO_INSURANCE)

```

```

        continue;

        game[0].board[i].property.insuranceRoundsLeft--;

        if(game[0].board[i].property.insuranceRoundsLeft == INSURANCE_EXPIRY_WARNING)
        {
            printf("\nInsurance policy on %s expires in %d rounds.\n",
                game[0].board[i].name, INSURANCE_EXPIRY_WARNING);
        }

        if(game[0].board[i].property.insuranceRoundsLeft <= 0)
        {
            printf("\nInsurance policy on %s has expired.\n",
                game[0].board[i].name);

            game[0].board[i].property.insurance = NO_INSURANCE;
        }
    }
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Board index.

Logic

- Only affects properties holding a policy.

4.10 market.c

The property market: boom/decline reviews every 10 rounds, regional development cards every 15 rounds, and the Rule-LK 36 market conditions report.

File header (includes / comments)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

```

initMarket

```
void initMarket(GameState game[])
```

Resets market state: all group cooldowns 0, no last boom/decline.

Actual source code

```

void initMarket(GameState game[])
{
    int i;

    for(i = 0; i < NO_GROUP; i++)
        game[0].economy.groupCooldownUntilRound[i] = 0;

    game[0].economy.lastBoomGroup = -1;
    game[0].economy.lastDeclineGroup = -1;
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Group index.

Logic

- Set by startGame right after board/players/economy initialization.

groupName

```
void groupName(PropertyGroup group, char buffer[])
```

Writes the display name of a colour group.

Actual source code

```
void groupName(PropertyGroup group, char buffer[])
{
    if(group == BROWN)
        strcpy(buffer, "Brown");
    else if(group == LIGHT_BLUE)
        strcpy(buffer, "Light Blue");
    else if(group == PINK)
        strcpy(buffer, "Pink");
    else if(group == ORANGE)
        strcpy(buffer, "Orange");
    else if(group == RED)
        strcpy(buffer, "Red");
    else if(group == YELLOW)
        strcpy(buffer, "Yellow");
    else if(group == GREEN)
        strcpy(buffer, "Green");
    else if(group == DARK_BLUE)
        strcpy(buffer, "Dark Blue");
    else
        strcpy(buffer, "Unknown");
}
```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
group	The group.
buffer	Destination.

Local variables

Name	Meaning
(none)	

Logic

- Handles all 8 real groups plus Unknown.

pickEligibleGroup

```
PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
```

Picks a random group not on cooldown and not the groups to avoid.

Actual source code

```
/* Picks a random group that isn't on cooldown and isn't one of the
   two groups to avoid (Rule-LK 30, 33). */
PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
{
    int candidates[NO_GROUP];
    int count;
    int g;

    count = 0;

    for(g = 0; g < NO_GROUP; g++)
    {
        if(g == avoid1 || g == avoid2)
            continue;

        if(game[0].economy.groupCooldownUntilRound[g] > currentRound)
            continue;

        candidates[count] = g;
        count++;
    }

    if(count == 0)
        return (PropertyGroup)((avoid1 + 1) % NO_GROUP); /* rare fallback */

    return (PropertyGroup)candidates[rand() % count];
}
```

Return value

Name	Meaning
returns	PropertyGroup.

Parameters

Name	Meaning
currentRound	For cooldown comparison.
avoid1	Group to avoid (or -1).
avoid2	Second group to avoid (or -1).

Local variables

Name	Meaning
candidates	Eligible groups.
count	Their count.
g	Loop index.

Logic

- Fallback when nothing eligible: (avoid1 + 1) % NO_GROUP, a rare defensive path.

reviewPropertyMarket

```
void reviewPropertyMarket(GameState game[], int currentRound)
```

Rule-LK 30-34: one group booms (+20% value, +25% rent, +15% mortgage/purchase, +10% construction) and a different one declines (-15% value, -20% rent, -10% mortgage, -25% auction), each for 10 rounds.

Actual source code

```

/* Rule-LK 30-34: every 10 rounds, one group booms and a different
   one declines, each for 10 rounds. All effects are temporary
   modifiers (Rule-LK 34) so no base value or cost is ever changed. */
void reviewPropertyMarket(GameState game[], int currentRound)
{
    PropertyGroup boomGroup;
    PropertyGroup declineGroup;
    char boomName[12];
    char declineName[12];

    boomGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastBoomGroup, -1);
    declineGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastDeclineGroup, boomGroup);

    groupName(boomGroup, boomName);
    groupName(declineGroup, declineName);

    printf("\n=== Property Market Review ===\n");
    printf("Market Boom : %s (values +20%%, rent +25%%) for 10 rounds.\n", boomName);
    printf("Market Decline : %s (values -15%%, rent -20%%) for 10 rounds.\n", declineName);

    addSourcedModifier(game, MOD_GROUP_VALUE, boomGroup, -1, 120, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_GROUP_RENT, boomGroup, -1, 125, 10, SRC_BOOM);

    addSourcedModifier(game, MOD_GROUP_VALUE, declineGroup, -1, 85, 10, SRC_DECLINE);
    addSourcedModifier(game, MOD_GROUP_RENT, declineGroup, -1, 80, 10, SRC_DECLINE);

    /* Rule-LK 31/32/34: the boom group's mortgage values rise by 15%,
       direct purchase prices rise by 15% and construction costs rise
       by 10%; the decline group's mortgage values fall by 10% and
       auction starting prices fall by 25%. All are scoped to the
       affected group. */
    addSourcedModifier(game, MOD_MARKET_MORTGAGE, boomGroup, -1, 115, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_PURCHASE_PRICE, boomGroup, -1, 115, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_CONSTRUCTION, boomGroup, -1, 110, 10, SRC_BOOM);

    addSourcedModifier(game, MOD_MARKET_MORTGAGE, declineGroup, -1, 90, 10, SRC_DECLINE);
    addSourcedModifier(game, MOD_AUCTION_PRICE, declineGroup, -1, 75, 10, SRC_DECLINE);

    game[0].economy.groupCooldownUntilRound[boomGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;
    game[0].economy.groupCooldownUntilRound[declineGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;

    game[0].economy.lastBoomGroup = boomGroup;
    game[0].economy.lastDeclineGroup = declineGroup;
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
currentRound	Round number for cooldowns.

Local variables

Name	Meaning
boomGroup	Chosen boom group.
declineGroup	Chosen decline group.

boomName, declineName Formatted names.

Logic

- All effects are temporary sourced modifiers (SRC_BOOM / SRC_DECLINE) - no base value is ever changed.
- Sets both groups' cooldowns to currentRound + 30 and remembers the last picks so they can't repeat.

drawRegionalCard

```
void drawRegionalCard(GameState game[])
```

Table 4: one of 12 regional development cards every 15 rounds, active for 15, all as SRC_REGIONAL modifiers.

Actual source code

```
/* Table 4: one card every 15 rounds, active for 15 rounds. */
void drawRegionalCard(GameState game[])
{
    int choice;

    choice = rand() % REGIONAL_CARD_COUNT;

    printf("\n=== Regional Development Card ===\n");

    switch(choice)
    {
        case 0:
            printf("Southern Tourism Boom : Galle Fort, Unawatuna and "
                "Hikkaduwa rental income +40%%.\n");
            addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 140, 15, SRC_REGIONAL);
            break;

        case 1:
            printf("Port City Expansion : Pettah and Maradana values +25%%.\n");
            addSourcedModifier(game, MOD_GROUP_VALUE, BROWN, -1, 125, 15, SRC_REGIONAL);
            addSourcedModifier(game, MOD_INDEX_VALUE, -1, 5, 125, 15, SRC_REGIONAL);    /* Colombo Fort Station
*/
            break;

        case 2:
            printf("IT Industry Growth : Maharagama, Nugegoda and "
                "Kottawa values +20%%.\n");
            addSourcedModifier(game, MOD_GROUP_VALUE, PINK, -1, 120, 15, SRC_REGIONAL);
            break;

        case 3:
            printf("Northern Development Programme : Jaffna Town, Nallur "
                "and Trincomalee values +30%%.\n");
            addSourcedModifier(game, MOD_GROUP_VALUE, GREEN, -1, 130, 15, SRC_REGIONAL);
            break;

        case 4:
            printf("Tea Export Boom : Nuwara Eliya value +35%%.\n");
            addSourcedModifier(game, MOD_INDEX_VALUE, -1, 37, 135, 15, SRC_REGIONAL);
            break;

        case 5:
            printf("Airport Expansion : Negombo, Katunayake and "
                "Ja-Ela rents +30%%.\n");
            addSourcedModifier(game, MOD_GROUP_RENT, ORANGE, -1, 130, 15, SRC_REGIONAL);
            break;

        case 6:
```

```

        printf("University City Growth : Peradeniya and "
               "Kandy City values +20%%.\n");
        addSourcedModifier(game, MOD_GROUP_VALUE, RED, -1, 120, 15, SRC_REGIONAL);
        break;

    case 7:
        printf("Beach Pollution : Southern coastal rents -30%%.\n");
        addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 70, 15, SRC_REGIONAL);
        break;

    case 8:
        printf("Flood Damage : Low-lying coastal properties lose 20%% value.\n");
        addSourcedModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 80, 15, SRC_REGIONAL);
        break;

    case 9:
        printf("Transport Strike : Railway revenue reduced by 40%%.\n");
        addSourcedModifier(game, MOD_RAIL_RENT, -1, -1, 60, 15, SRC_REGIONAL);
        break;

    case 10:
        printf("Electricity Tariff Increase : Utility rent +25%%.\n");
        addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 125, 15, SRC_REGIONAL);
        break;

    case 11:
        printf("Water Shortage : Utility revenue +20%%.\n");
        addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 120, 15, SRC_REGIONAL);
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 13, 90, 15, SRC_REGIONAL);
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 29, 90, 15, SRC_REGIONAL);
        break;

    default:
        break;
}
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
choice	rand() % 12.

Logic

- Cards boost or cut province values/rents (Southern Tourism, Port City, IT Industry, Northern Programme, Tea Boom, Airport, University, Beach Pollution, Flooding, Transport Strike, Electricity Tariff, Water Shortage).
- Some cards also add per-index modifiers (e.g. Colombo Fort station, tea-export square, water squares).

printSectionHeader

```
void printSectionHeader(char title[])
```

Prints a report section title and separator.

Actual source code

```

/* Prints a section title followed by a separator line. */
void printSectionHeader(char title[])
{
    printf("%s\n", title);
    printf("-----\n");
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
title	Heading text.

Local variables

Name	Meaning
(none)	

Logic

- Small formatting helper used by the conditions report.

printBoomDecline

```
void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
```

Prints one Market Boom / Market Decline block.

Actual source code

```

/* Prints a Market Boom or Market Decline block. */
void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
{
    printSectionHeader(title);
    printf("%s (%+d%)\n", groupNameBuffer, percent - 100);
    printf("Rounds Remaining : %d\n", roundsLeft);
}

```

Return value

Name	Meaning
returns	void.

Parameters

Name	Meaning
title	Boom or Decline.
groupNameBuffer	Group name.
percent	Value multiplier.
roundsLeft	Rounds remaining.

Local variables

Name	Meaning
(none)	

Logic

- Shows the signed percent (percent - 100) and the time left.

describeRegionalCondition

```
int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
```

Writes one regional card effect line; returns 1 if a line was produced.

Actual source code

```
/* Writes one Regional Development card line into out. Returns 1 if
the modifier has a line to show, 0 otherwise. */
int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
{
    char name[48];

    if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
    {
        if(mod.group >= 0 && mod.group < NO_GROUP)
            groupName((PropertyGroup)mod.group, name);
        else
            strcpy(name, "Unknown");

        if(mod.type == MOD_GROUP_VALUE)
            sprintf(out, "%s (%+d%%)", name, mod.percent - 100);
        else
            sprintf(out, "%s rents (%+d%%)", name, mod.percent - 100);
    }
    else if(mod.type == MOD_INDEX_VALUE)
    {
        sprintf(out, "%s (%+d%%)",
            game[0].board[mod.index].name, mod.percent - 100);
    }
    else if(mod.type == MOD_RAIL_RENT)
    {
        sprintf(out, "Railway rents (%+d%%)", mod.percent - 100);
    }
    else if(mod.type == MOD_UTIL_RENT)
    {
        sprintf(out, "Utility rents (%+d%%)", mod.percent - 100);
    }
    else
    {
        return 0;
    }

    return 1;
}
```

Return value

Name	Meaning
returns	1 = wrote a line, 0 = not a printable type.

Parameters

Name	Meaning
mod	A modifier.
out	Destination.

Local variables

Name	Meaning
name	Group or square name.

Logic

- Handles group value/rent, index value, rail and utility rent. Called for every SRC_REGIONAL modifier.

printRegionalSection

```
int printRegionalSection(GameState game[])
```

Prints all active Regional Development effects under one header.

Actual source code

```
/* Prints the "Regional Development" section. Returns 1 if at least
   one card effect was shown. */
int printRegionalSection(GameState game[])
{
    int i;
    int shown;

    shown = 0;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        ActiveModifier mod;
        char line[100];

        mod = game[0].economy.modifiers[i];

        if(mod.source != SRC_REGIONAL)
            continue;

        if(!describeRegionalCondition(game, mod, line))
            continue;

        if(!shown)
        {
            printSectionHeader("Regional Development");
            shown = 1;
        }

        printf("%s\n", line);
        printf("Rounds Remaining : %d\n", mod.roundsLeft);
    }

    return shown;
}
```

Return value

Name	Meaning
returns	1 if anything was printed.

Local variables

Name	Meaning
i	Modifier index.
shown	Whether the header was printed.
mod	Current modifier.
line	Formatted line.

Logic

- Collects SRC_REGIONAL modifiers with a printable describeRegionalCondition.

describeOtherCondition

```
int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
```

Writes one 'Other Active Conditions' line; returns 1 if written.

Actual source code

```
/* Writes one "Other Active Conditions" line into out. Returns 1 if
the modifier has a line to show, 0 otherwise. */
int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
{
    char name[32];

    if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
    {
        groupName((PropertyGroup)mod.group, name);

        if(mod.type == MOD_GROUP_VALUE)
            sprintf(out, "%s : values x%d%% (%d rounds remaining)",
                    name, mod.percent, mod.roundsLeft);
        else
            sprintf(out, "%s : rents x%d%% (%d rounds remaining)",
                    name, mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_INDEX_VALUE)
    {
        sprintf(out, "%s : value x%d%% (%d rounds remaining)",
                game[0].board[mod.index].name, mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_RAIL_VALUE)
    {
        sprintf(out, "Railway values : x%d%% (%d rounds remaining)",
                mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_RAIL_RENT)
    {
        sprintf(out, "Railway rents : x%d%% (%d rounds remaining)",
                mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_UTIL_RENT)
    {
        sprintf(out, "Utility rents : x%d%% (%d rounds remaining)",
                mod.percent, mod.roundsLeft);
    }
    else
    {
        return 0;
    }

    return 1;
}
```

Return value

Name	Meaning
returns	1 = wrote a line, 0 = not printable.

Parameters

Name	Meaning
mod	A modifier.

out	Destination.
-----	--------------

Local variables

Name	Meaning
name	Group or square name.

Logic

- Handles group value/rent, index value, rail value/rent and utility rent.

printOtherConditions

```
void printOtherConditions(GameState game[])
```

Prints still-running event/regulation (SRC_GENERAL) modifiers.

Actual source code

```
/* Prints the "Other Active Conditions" section (events and
   government regulations still running). */
void printOtherConditions(GameState game[])
{
    int i;
    int shown;

    shown = 0;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        ActiveModifier mod;
        char line[100];

        mod = game[0].economy.modifiers[i];

        if(mod.source != SRC_GENERAL)
            continue;

        if(!describeOtherCondition(game, mod, line))
            continue;

        if(!shown)
        {
            printSectionHeader("Other Active Conditions");
            shown = 1;
        }

        printf("%s\n", line);
    }
}
```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Modifier index.
shown	Header printed flag.
mod	Current modifier.
line	Formatted line.

Logic

- Used at the end of the market report.

displayMarketConditions

```
void displayMarketConditions(GameState game[])
```

Rule-LK 36: the end-of-round market report - boom, decline, regional development, other conditions, inflation, loan interest and tax rate.

Actual source code

```
/* Rule-LK 36: shows the currently active market conditions at the
   end of every round, in the Section 5 format. */
void displayMarketConditions(GameState game[])
{
    int i;
    int shown;

    printf("\n=====\\n");
    printf("Current Market Conditions\\n");
    printf("=====\\n");

    shown = 0;

    /* Market Boom (Rule-LK 31) */
    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        char name[32];

        if(game[0].economy.modifiers[i].source != SRC_BOOM ||
           game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
            continue;

        groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
        printBoomDecline("Market Boom", name,
                        game[0].economy.modifiers[i].percent,
                        game[0].economy.modifiers[i].roundsLeft);

        shown = 1;
        break;
    }

    /* Market Decline (Rule-LK 32) */
    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        char name[32];

        if(game[0].economy.modifiers[i].source != SRC_DECLINE ||
           game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
            continue;

        groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
        printBoomDecline("Market Decline", name,
                        game[0].economy.modifiers[i].percent,
                        game[0].economy.modifiers[i].roundsLeft);

        shown = 1;
        break;
    }

    /* Regional Development Card effects (Table 4) */
    if(printRegionalSection(game))
        shown = 1;
}
```

```

    if(!shown)
        printf("No active market booms, declines or regional conditions right now.\n");

    /* Other active event/regulation effects */
    printOtherConditions(game);

    printSectionHeader("Inflation");
    printf("%d%%\n", game[0].economy.inflationRate);

    printSectionHeader("Current Loan Interest");
    printf("%d%%\n", game[0].economy.loanInterestRate);

    printSectionHeader("Current Income Tax Rate");
    printf("%d%%\n", game[0].economy.incomeTaxRate);

    printf("=====\n");
}

```

Return value

Name	Meaning
returns	void.

Local variables

Name	Meaning
i	Modifier index.
shown	Whether anything was printed.
name	Group name.

Logic

- Picks the boom and decline from SRC_BOOM/SRC_DECLINE MOD_GROUP_VALUE modifiers, prints regional + other sections, then inflation/interest/tax always.

4.11 main.c

The entry point. Creates the single GameState and hands control to startGame.

File header (includes / comments)

```

#include <stdio.h>
#include "types.h"
#include "functions.h"

```

main

```
int main(void)
```

The only place a GameState is ever created (game[1] = {0}), then startGame runs the whole simulation.

Actual source code

```

int main(void)
{
    /* the only place a GameState is ever created */
    GameState game[1] = {0};

    printf("=====\n");
    printf("        MONOPOLY-LK SIMULATION\n");
    printf("=====\n\n");

    startGame(game);
}

```

```
    return 0;
}
```

Return value

Name	Meaning
returns	0 on success.

Local variables

Name	Meaning
game	The single GameState (array-of-one idiom).

Logic

- Prints the banner, then calls `startGame(game)`.
- The array-of-one idiom means every function receives a pointer that is used with plain array syntax (`game[0].field`), avoiding pointer arithmetic.